ZEDPATH

Team Kestrel · IntelliCon '26 Buildathon

ZP-DOC-05

Data Design

Conceptual EER model, relational mapping, normalisation and the Cloudflare D1 physical schema

Version	0.9
Status	In review
Date	11 October 2026
Classification	Public
Document owner	B.M.P Banneka, Product Owner

© 2026 Team Kestrel. Published under the classification shown above.

Uncontrolled when printed or exported. The controlled version is the Markdown source in the repository: docs/05-data-design/ZP-DOC-05_Data-Design.md

Document control

Document information

Document ID	ZP-DOC-05
Title	Data Design
Version	0.9
Status	In review
Classification	Public
Document owner	B.M.P Banneka, Product Owner
Author	Team Kestrel (drafted with Claude, AI coding partner)
Conforms to	Enhanced ER model and mapping algorithm (Elmasri and Navathe, 7th ed.); relational normalisation to BCNF/4NF; ZP-DOC-00
Source file	docs/05-data-design/ZP-DOC-05_Data-Design.md
Repository	https://github.com/PraveenBanneka/zedpath

Revision history

Version	Date	Author	Description of change
0.9	5 Oct 2026	Team Kestrel	First issue: four EER diagrams, rule grammar, mapping, normalisation, D1 schema validated against the real UGC 2025/26 and 2024/25 data; submitted for approval
0.9	5 Oct 2026	Team Kestrel	Revised in review for CR-001 (student accounts, approved by the product owner): Section 2.5 EER model, mapping, normalisation, security conventions, 7 tables (migrations 0002 and 0003), 1:1 cardinality in the generated logical diagrams, evidence
0.9	11 Oct 2026	Team Kestrel	Revised in review for CR-002 (Journey, Alerts, Groups; product owner's GO): Section 2.6, migrations 0005 and 0006, read-model decision for dates and notices
0.9	11 Oct 2026	Team Kestrel	Revised in review for CR-003 (compulsory account with contact details; product owner's decision): STUDENT_CONTACT, migration 0007, the "username only" statement of CR-001 withdrawn
0.9	11 Oct 2026	Team Kestrel	Revised in review for CR-004 (Ask answer feedback): ANSWER_FEEDBACK, migration 0008
0.9	11 Oct 2026	Team Kestrel	Revised in review for CR-005 (group rules, product owner's request): GROUP_MEMBER.rules_version, migration 0009; labelled sample conversations as read-model data
0.9	11 Oct 2026	Team Kestrel	Revised in review for CR-006 (product owner's decisions of 11 Oct): rules accepted once per account (GROUP_RULES_ACCEPTANCE, GROUP_MEMBER.rules_version withdrawn), ACCOUNT.avatar, the Journey on the account (JOURNEY_COURSE, JOURNEY_NOTICE, JOURNEY_STEP), OFFERING populated; migration 0010

Version	Date	Author	Description of change
0.9	15 Oct 2026	Team Kestrel	Revised in review for CR-007 (product owner's decision of 15 Oct): demo accounts as ACCOUNT.demo_expires_at with a partial index; migration 0011
0.9	15 Oct 2026	Team Kestrel	University facts from the UGC 'Sri Lanka University Statistics 2024' added as read-model data (no D1 change)
0.9	15 Oct 2026	Team Kestrel	Security audit run-1 lead 3: GROUP_ROOM_SEEN (rooms an account opened) so account deletion purges every room; migration 0012

Review and approval

This document takes effect when the approver signs below. Until then its status is as shown on the cover.

Role	Name	Signature	Date
Reviewer	Claude (AI coding partner), consistency and traceability review		
Approver	B.M.P Banneka, Product Owner		

Contents

1 Introduction 6
1.1 Purpose 6
1.2 Scope 6
1.3 Intended audience 6
1.4 Relationship to other documents 6
1.5 Notation 6
2 Conceptual model (EER) 7
2.1 Offerings and cut-offs 7
2.2 Subjects and requirement rules 8
2.3 Provenance and trust 9
2.4 Journey and other routes 10
2.5 Student accounts (CR-001) 11
2.6 Journey, Alerts and Groups (CR-002) 11
2.7 Contact details (CR-003) 12
2.8 Answer feedback (CR-004) 13
2.9 Group rules and sample conversations (CR-005) 13
2.10 Journey, avatars and group rules on the account (CR-006) 13
2.11 Demo accounts (CR-007) 15
2.12 University facts from UGC statistics (read model) 15
2.13 Rooms an account opened (security audit, lead 3) 15
2.14 Entity catalogue 16
3 Rule grammar 18
4 Mapping the EER model to relations 19
5 Normalisation 23
5.1 From the source form to 1NF 23
5.2 Functional and multi-valued dependencies 23
5.3 2NF, 3NF and BCNF 23
5.4 Deliberate departures 24
6 Physical design for Cloudflare D1 25
6.1 Conventions 25
6.2 Indexes 25
6.3 Capacity against the free plan 26
7 Validation against real data 27
Appendix A: Traceability: requirements to tables 29
Appendix B: Physical schema (migration 0001) 30

Appendix C: Student accounts (migration 0003) 36
Appendix D: Contact details (migration 0007) 38
Appendix E: Answer feedback (migration 0008) 39
Appendix F: Journey, avatars and group rules on the account (migration 0010) 40
Appendix G: Rooms an account opened (migration 0012) 45
Appendix H: Demo accounts (migration 0011) 46
Appendix I: Group rules (migration 0009) 47
Appendix J: Groups (migration 0006) 48
Appendix K: Glossary 50
Appendix L: References 51

List of figures

Figure 1: EER model of institutions, courses, offerings, intake years, selection groups and cut-offs 8
Figure 2: EER model of streams, subjects and the recursive requirement-rule tree 9
Figure 3: EER model of cited facts, sources, revisions, AI extraction, review and mistake reports 10
Figure 4: EER model of deadlines, push reminders and the specialisation of other routes 10
Figure 5: EER model of optional student accounts: sign-in, sessions, recovery and the student's results 11
Figure 6: Logical schema, admission data (generated from the migration file) 21
Figure 7: Logical schema, trust, routes and journey data (generated from the migration file) 22
Figure 8: Logical schema, optional student accounts (generated from the migration files) 22

List of tables

Table 1: Position of this document in the ZedPath document set 6
Table 2: CR-006 changes to the conceptual model 14
Table 3: CR-007 changes 15
Table 4: Entities, their type, identifier and the requirements that justify them 16
Table 5: Rule node kinds and their meaning 18
Table 6: EER-to-relational mapping steps applied 19
Table 7: Mapping of the student-account model 20
Table 8: Dependencies identified in R1 23
Table 9: Controlled denormalisations and why they are safe 24
Table 10: Secondary indexes and the query each serves 25
Table 11: Data volume per intake year and free-plan headroom 26
Table 12: Schema validation results (5 October 2026) 27
Table 13: Student-account validation results (5 October 2026) 27
Table 14: Requirements and the tables that realise them 29
Table 15: Terms used in this document 50
Table 16: Sources referred to in this document 51

1 Introduction

1.1 Purpose

This document designs ZedPath's data in three steps, as a database engineer would: a **conceptual** model in Enhanced Entity-Relationship (EER) notation, its **logical** relational schema derived with the standard mapping algorithm and normalised, and the **physical** schema for Cloudflare D1. The physical schema is the real migration file the application uses, and it has been proved by loading the real UGC data into it.

1.2 Scope

All data held on the server for Releases R1 and R2. By default the student's own profile lives only on their device (NFR-030). Change request CR-001 (5 October 2026) adds an **optional** student account: a student who chooses to create one keeps the same profile on the server, under a username only, so it follows them between devices and can drive reminders. Section 2.5 models it. The preference list stays on the device.

1.3 Intended audience

Developers, reviewers and anyone assessing the data design.

1.4 Relationship to other documents

Table 1: Position of this document in the ZedPath document set

Document	Relationship
ZP-DOC-03 SRS	Parent. Business rules BR-001 to BR-043 and data-quality requirements NFR-050 to NFR-052 become entities and constraints
ZP-DOC-04 Use Case Model	Parent. Nouns in the use cases name the entities
ZP-DOC-06 Software Architecture	Child. Decides how the read path uses this schema within free-plan limits
`migrations/0001_initial_schema.sql`	Implementation. Appendix A reproduces it verbatim
`migrations/0002_reference_data.sql`, `0003_student_accounts.sql`	Implementation of CR-001. 0002 is generated reference data; Appendix B reproduces 0003 verbatim

1.5 Notation

Conceptual diagrams use Chen notation as extended by Elmasri and Navathe: rectangles are entities; double rectangles are **weak** entities; diamonds are relationships; double diamonds are **identifying** relationships; ovals are attributes, with keys underlined, double ovals for **multi-valued** and dashed ovals for **derived** attributes; bold lines show **total participation**; a circle marked *d* or *o* shows a **disjoint** or **overlapping specialisation**, and a circle marked *U* a **union category**. Partial keys of weak entities are named in the entity catalogue. Logical diagrams use crow's-foot (Information Engineering) notation.

2 Conceptual model (EER)

2.1 Offerings and cut-offs

The admission domain has a feature that a simple design misses: a **course of study** (for example, Management) is offered at several **institutions**; each pair is an **offering** with its own Uni-Code; an offering exists anew in each **intake year** with its own intake and requirements; and a few offerings split their seats into **selection groups** by stream or category, each with its own cut-offs. Loading the real 2025/26 cut-off table confirmed this: its 260 columns correspond to 255 Uni-Codes, because five Uni-Codes are split.

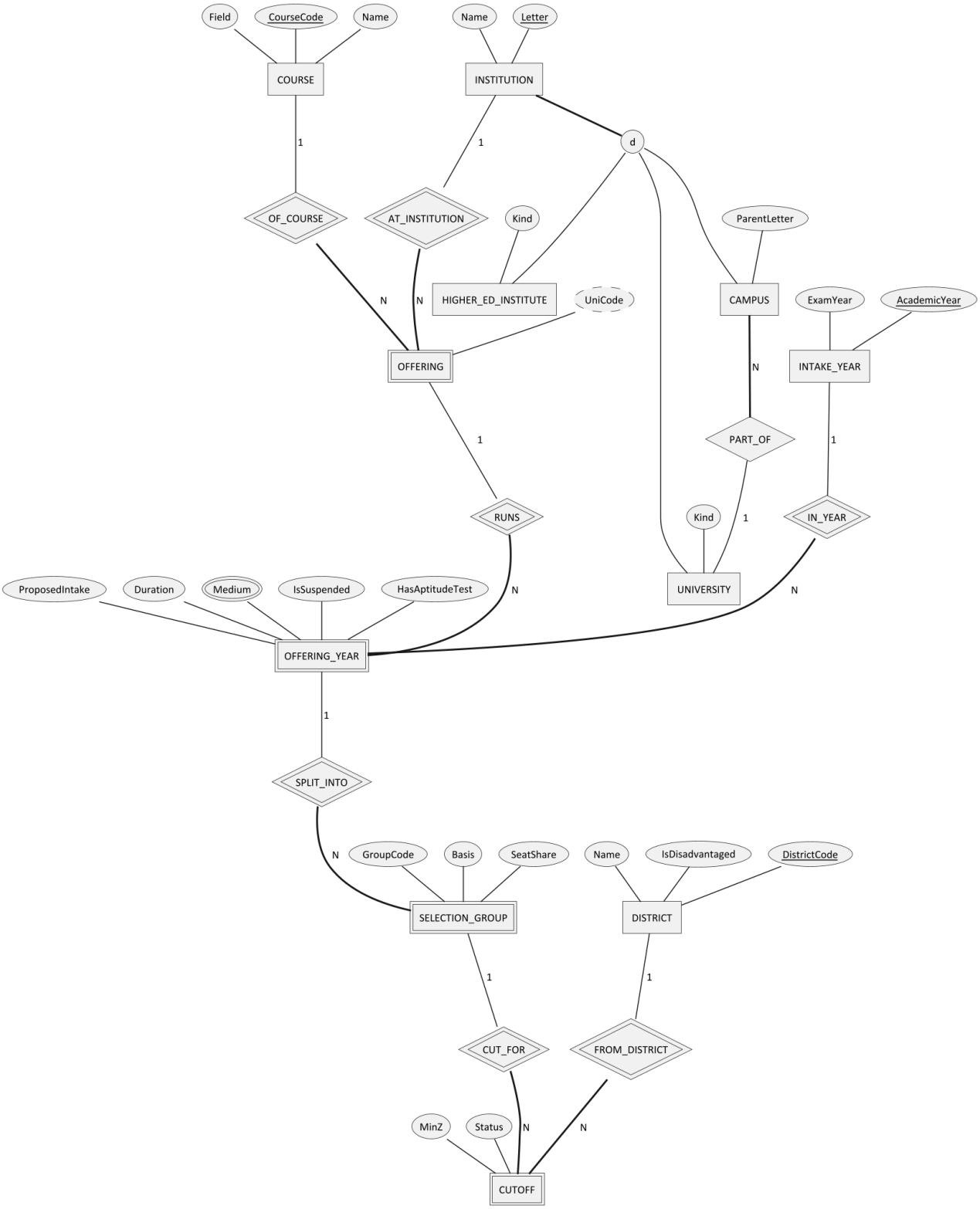


Figure 1: EER model of institutions, courses, offerings, intake years, selection groups and cut-offs

2.2 Subjects and requirement rules

Entry requirements follow twenty different patterns (BR-020). Instead of twenty columns or free text, every requirement is a **tree of rule nodes** (recursive relationship CHILD_OF) whose leaves test subjects. One structure serves state-university selection groups and other routes alike.

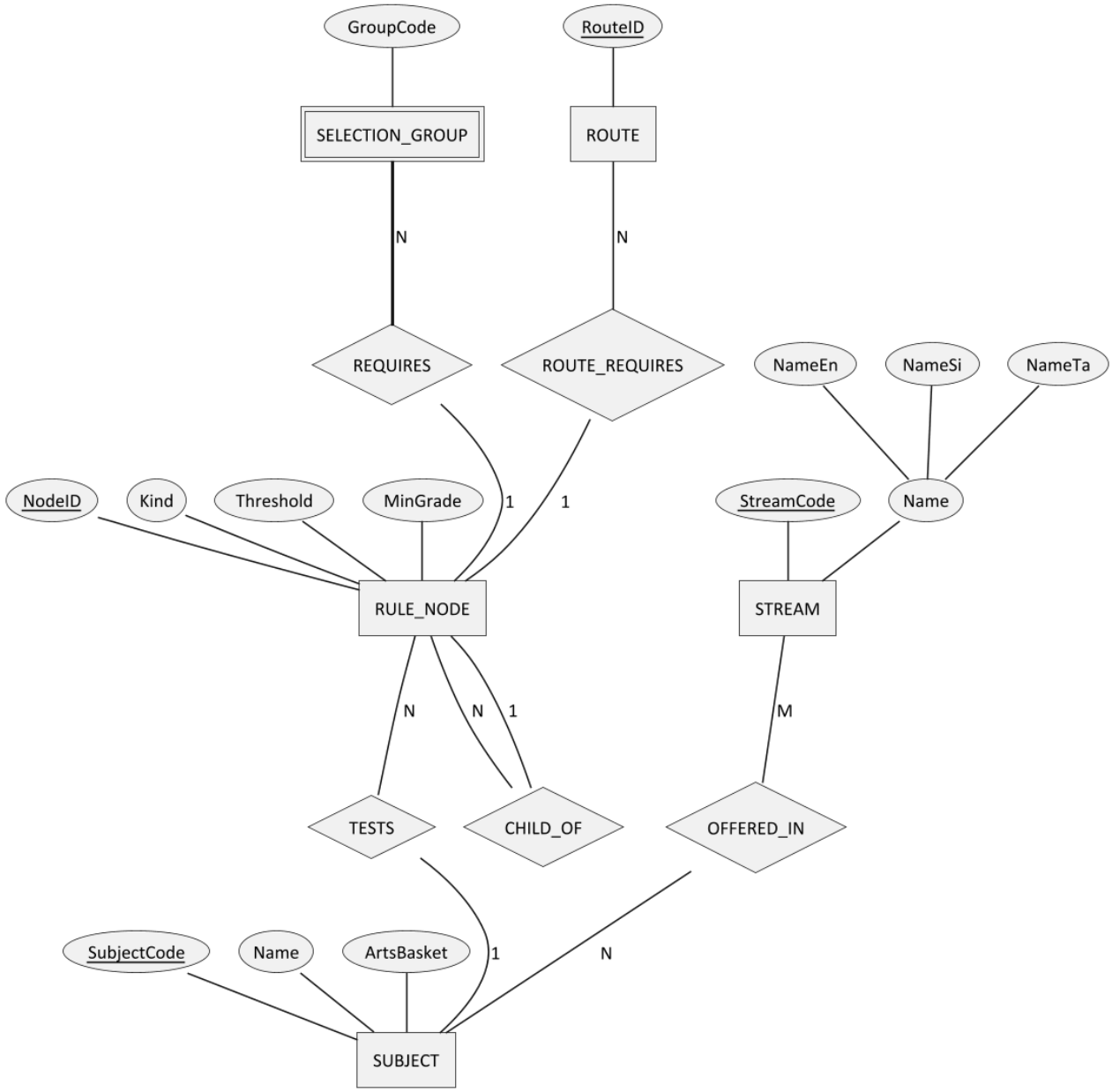


Figure 2: EER model of streams, subjects and the recursive requirement-rule tree

2.3 Provenance and trust

Every published fact must cite its source (BO-2). Cut-offs, offering details, rules, deadlines and routes are different entity types, yet each needs citations, revisions and mistake reports. The model makes them members of one **union category**, CITED_FACT, so these trust features are designed once.

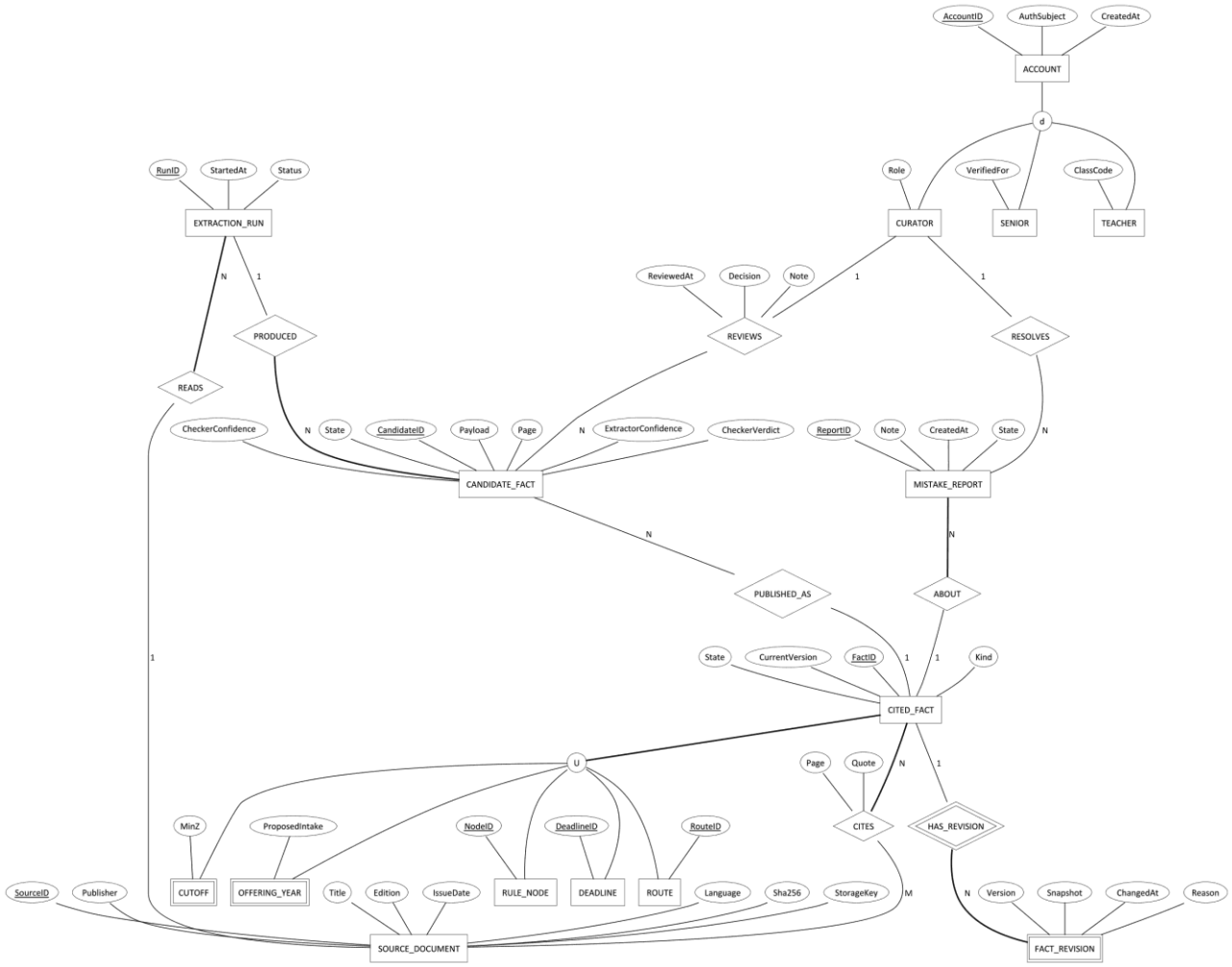


Figure 3: EER model of cited facts, sources, revisions, AI extraction, review and mistake reports

2.4 Journey and other routes

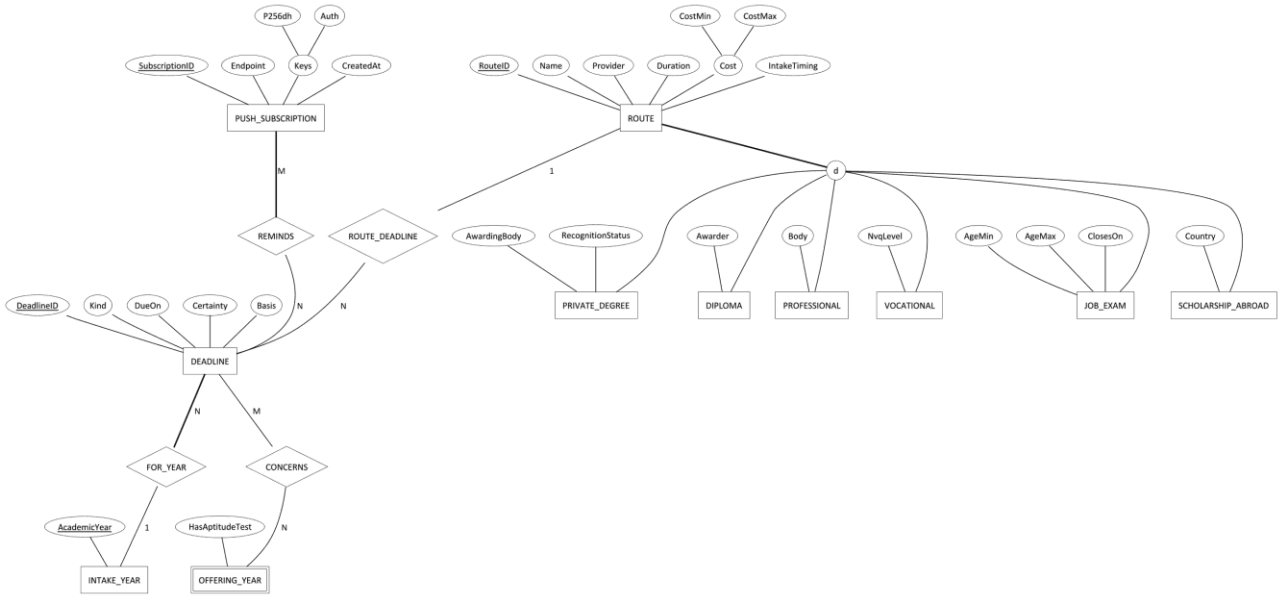


Figure 4: EER model of deadlines, push reminders and the specialisation of other routes

2.5 Student accounts (CR-001)

A student account is optional. ACCOUNT, already the superclass of the staff roles, gains a fourth subclass, STUDENT. Each way of signing in is its own weak entity of ACCOUNT, so a later identity provider (for example Google sign-in) adds an entity without touching the others. A password is never stored: PASSWORD_LOGIN holds a salt and a *verifier*, a hash of a key the student's device derives from the password. A STUDENT's results are the attributes the device already keeps: a Z-score, the stream sat (SAT_IN), the district sat from (SAT_FROM), and three subjects with grades (TOOK, whose attributes are Grade and Position). Achievements are a weak entity with partial key Position; interests are a multi-valued attribute. CR-001 stored no name, email address or telephone number; CR-003 (below) withdraws that and adds them, with consent, in a table of their own.

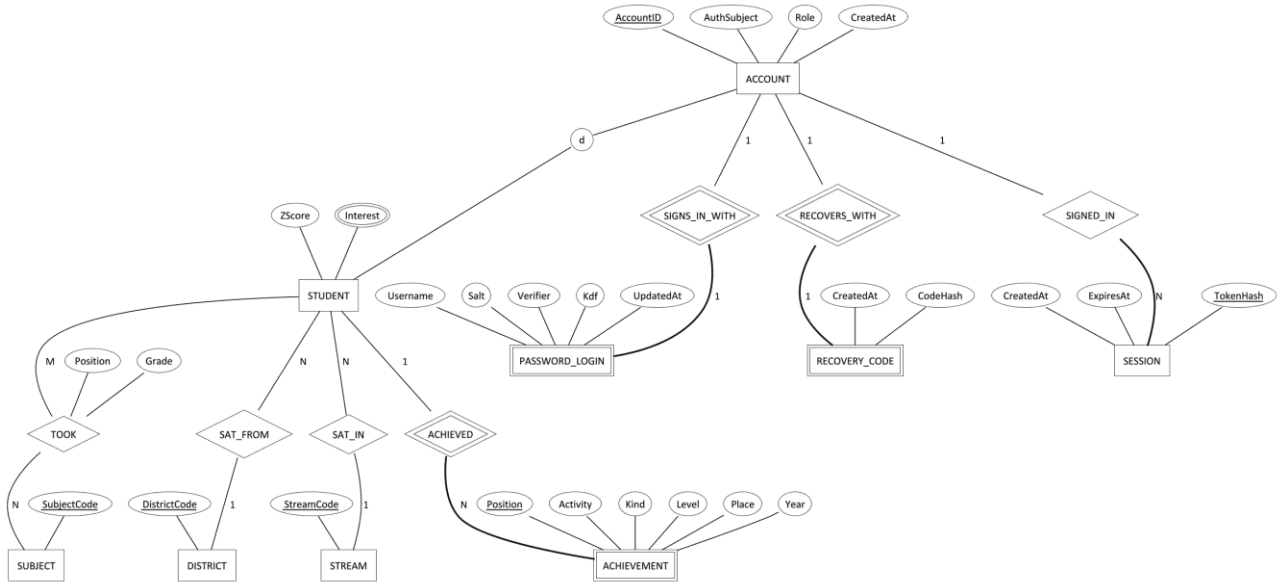


Figure 5: EER model of optional student accounts: sign-in, sessions, recovery and the student's results

2.6 Journey, Alerts and Groups (CR-002)

Where each kind of data lives. CR-002 adds three features. Each puts its data where the existing design already puts that kind of data, rather than adding a new store:

Data	Where	Why
UGC admission steps (order, rule, handbook page) and dated notices (Gazette job exams, intakes)	Bundled read model `worker/generated/journey-2025-2026.json`, built by `tools/journey/build-journey.ts` from `data/journey/` and `data/alerts/`	Reference facts, like cut-offs and routes (ADR-002): no database reads per request. The build refuses to write unless every quote is found verbatim in its source
A student's saved dates, date of birth, seen alerts, scheduled reminder ids	The student's phone only	Personal and only needed on that phone; nothing stored without asking (NFR-030). Eligibility is checked on the phone
Course groups, members, senior invites, reports, blocks	D1, migration 0006	Shared between accounts, so it must be on the server, enforced by keys and CHECKs
Group messages	Each group's GroupRoom Durable Object (its own SQLite storage)	High write rate per group, never joined with other data; keeps chat traffic off D1

The `deadline` table of migration 0001 remains the system-of-record design for dates. It is not loaded in this release: the UGC sets its dates by press notice each year, so the steps carry the official *rule* (for example, "within two weeks of the closing date", p.116) and dated items come from cited notices. No date is invented.

Verified seniors reuse the existing model. ACCOUNT already has the disjoint subclass SENIOR with the attribute VerifiedFor (`account.verified_for_letter`), and the 0003 triggers already keep student data off non-student accounts. A SENIOR_INVITE (one-time code, stored only as its SHA-256) names the course, institution and study year the product owner checked; redeeming it at sign-up creates the SENIOR account and marks the invite in one atomic batch. A trigger allows redemption only by a SENIOR account of the invite's institution, and only once.

Reference rows for foreign keys. Migration 0005 loads the 121 courses and 20 institutions of 2025/26 (generated by `tools/seed/courses-sql.ts` from the official Uni-Code list), so SENIOR_INVITE and COURSE_GROUP reference them.

Table (0006)	Key	Constraints and indexes	Deleting an account
senior_invite	invite_id; code_hash UNIQUE; redeemed_by UNIQUE	FK course, institution; CHECK study_year 1 to 6, expires after created, redeemed_by and redeemed_at both set or both null; trigger on redemption	Its redeemed invite is deleted (CASCADE)
course_group	group_id	kind COURSE (course_code) or BATCH (stream_code + district_code), enforced by three CHECKs; one group per course and per batch by two partial UNIQUE indexes (SQLite treats NULLs as distinct in a plain UNIQUE)	Unaffected
group_member	(group_id, account_id)	FK both, CASCADE; index on account_id	Memberships deleted
message_report	(group_id, message_id, reporter_id)	one report per member per message; reason CHECK in four values; message_id CHECK 32 hex	Reports deleted
account_block	(blocker_id, blocked_id)	CHECK not self; index on blocked_id	Blocks both ways deleted

Messages are outside D1's cascade, so deleting an account also tells every group it was in to delete that account's messages (tested). All tables are in BCNF: every determinant is a candidate key; derived values (member counts, the senior badge text) are computed, never stored.

2.7 Contact details (CR-003)

Decision. On 11 October 2026 the product owner made an account compulsory at the start, with full name, email, mobile number and date of birth (the birthday moves here from the Alerts screen, where it stayed on the phone). This reverses the CR-001 promise "a username only". The README, the business case and the app's privacy text now say exactly what is collected and why.

Where it lives. A weak entity CONTACT of ACCOUNT, 1:1 and total for accounts made after CR-003, mapped to its own table `student_contact` rather than columns on `account`: `account` is the identity table shared by every role, and personal contact data kept apart can be audited, exported or deleted on its own. It cascades with the account.

Attribute	Rule (CHECK in migration 0007, same rule in `shared/contact.ts` for the phone and the API)
full_name	2 to 80 characters after trimming
email	lower-case, an address shape, at most 254 characters; **not unique** (a uniqueness error would reveal that an address is registered)
mobile	Sri Lankan mobile in E.164 form `+947XXXXXXXX`; **not unique** (siblings may share a parent's number)
birth_date	a real calendar date; the API accepts ages 13 to 70
consent_version, consent_at	the privacy wording agreed to and when; the CHECK lists the versions that exist
email_verified_at, mobile_verified_at	NULL until verification exists (no sending domain or SMS service yet); reset when the value changes

Accounts made before CR-003 have no row; the app asks them to complete it before anything else (`PUT /api/me/contact`). Safe default: no existing row is invented or guessed. Students under 18 confirm that a parent or guardian agrees (Personal Data Protection Act, No. 9 of 2022). The table is in BCNF: the only determinant is the key.

2.8 Answer feedback (CR-004)

"Helpful" and "Report a wrong answer" under an Ask ZedPath answer are stored in `answer_feedback` (migration 0008). Not `mistake_report`: that table reports a stored, cited fact (its `fact_id` is required), and an AI answer is not one. A row holds the rating, the question, the answer, the handbook pages it cited (JSON, checked by `json_valid`), an optional note, the app language and a review state. It holds nothing that identifies the student: no account, device or IP. The screen says the question and answer are sent before the student sends a report.

2.9 Group rules and sample conversations (CR-005)

A member accepts the group rules before taking part. The accepted version is a fact about one membership, so it is a column on `group_member` (`rules_version`, migration 0009) rather than a new table: one value per (group, member), fully dependent on the key, so the table stays in BCNF. A CHECK lists the versions that exist (`'2026-10-11'`); a new version of the rules adds a value and asks members again. NULL means joined before the rules existed. Server enforcement: the Worker refuses a join without the current version and keeps the live room closed for a member whose row is not on it (`worker/groups.ts`). Rule 3 (no phone numbers or email addresses) is checked on every message by the group's room. *Superseded by CR-006:* acceptance moved from the membership to the account; migration 0010 carries `group_member.rules_version` over and leaves the column unused until a later migration drops it.

Sample conversations, which show a new student how a group works, are **not stored**. Like the Journey steps they are curated read-model data in code (`shared/group-samples.ts`), added by the room in front of the real messages. Every sample is marked `sample`, its author starts with "Sample", no badge says "verified", and its id is not a message id, so it cannot be reported, blocked or mistaken for a member (trust rule: sample data is labelled as sample; pinned by tests).

2.10 Journey, avatars and group rules on the account (CR-006)

The product owner decided on 11 October 2026: group rules are accepted **once per account**; each account may pick a **preset avatar** (no photo upload); the **Journey is saved on the account**; and a Journey step can be ticked **only once its window is open**, which the server checks. Migration 0010 implements the design below.

Table 2: CR-006 changes to the conceptual model

Concept	EER construct	Relation (migration 0010)	Key and constraints
An account accepts a version of the group rules	M:N relationship ACCEPTS between ACCOUNT and the (enumerated) RULES_VERSION, with attribute *accepted_at*	`group_rules_acceptance`	PK (account_id, rules_version); FK account ON DELETE CASCADE; CHECK on the versions that exist
An account has a picture	Single-valued attribute of ACCOUNT	`account.avatar`	CHECK: one of 12 preset codes or NULL (initials)
A student follows a course	M:N relationship FOLLOWS_COURSE between ACCOUNT and OFFERING, attribute *followed_at*	`journey_course`	PK (account_id, course_code, institution_letter); composite FK to `offering`; FK account CASCADE
A student follows a notice or other path, and may say they applied	M:N relationship FOLLOWS_NOTICE between ACCOUNT and ROUTE, attributes *followed_at*, *applied_at*	`journey_notice`	PK (account_id, route_id); CHECK on the id's shape; CHECK applied_at >= followed_at
A student ticks a Journey step	M:N relationship COMPLETES between ACCOUNT and the (enumerated) JOURNEY_STEP, attribute *done_at*	`journey_step`	PK (account_id, step_id); CHECK on the step ids

Mapping. Every new relationship is many-to-many with its own attributes, so each maps to its own relation whose key is the pair of participating keys (mapping step 5). The avatar is a single-valued attribute of ACCOUNT and stays on it (step 1). The acceptance is no longer a column of GROUP_MEMBER because it does not depend on the group: under CR-005 the same fact was repeated once per group the student joined, a partial-dependency redundancy that CR-006 removes.

Normalisation. In each new relation the only non-key attributes (`accepted_at`, `followed_at`, `applied_at`, `done_at`) depend on the whole key and on nothing else, so all three are in BCNF; `account.avatar` depends on `account_id` alone. There are no multi-valued dependencies: courses, notices and steps are independent facts kept in separate relations rather than one wide table (4NF).

Reusing existing structure. `offering` was defined in 0001 but left empty because the read model serves the course data. 0010 fills it with the 255 Uni-Codes of 2025/26 (handbook pages 142 to 148, generated by `tools/seed/cr006-sql.ts` from the same official list as 0005), so a followed course is a real foreign key, not free text. The route catalogue, by contrast, stays read-model data compiled into the Worker (the D1 `route` table uses integer ids and is not populated), so `journey_notice.route_id` is checked by shape in D1 and against the catalogue by the Worker: a documented departure in the table below.

Business rules on the server. The step list (three preparation steps, open at any time, and the five UGC steps) is fixed in `shared/journey.ts` and mirrored by the CHECK on `journey_step.step_id`; a test inserts every step id the code knows, so the two cannot drift. A UGC step opens on the date of the UGC press notice (`opensOn` in the journey read model); while no date is announced it cannot be ticked, because ZedPath never invents a date. "I applied" on a followed notice can be set only once the notice has opened. The Worker checks both against the read model, using the Sri Lanka date (UTC+05:30), before writing (`worker/my-journey.ts`); the phone shows the same rule.

Data migration and safe defaults. Rules accepted under 0009 are carried into `group_rules_acceptance` with the time of the account's first join (tested on a copy of the database as it was before 0010). `group_member.rules_version` is no longer read or written but is kept for now (expand, then contract): the previous release still writes it, so a rollback stays safe; a later migration drops it once this release is stable. No avatar means initials, as before. A new account's Journey is empty: nothing is ticked for the student. Journey items kept on a phone before CR-006 are copied to the account by the app on first use.

Privacy. The members list of a group (members only) returns usernames, avatars and senior badges, never the name, email or mobile in `student_contact`. Every new relation cascades on account deletion. In each group's room, a message keeps the author's avatar as it was when posted, a snapshot like the author name (a deliberate departure).

2.11 Demo accounts (CR-007)

The product owner asked for a one-tap demo on the Welcome screen (15 October 2026), choosing a **private copy per visitor** over one shared account. Each tap creates a throwaway account holding the sample student; it expires 24 hours later and is deleted on a later demo tap (the free plan has no scheduled job).

Where the fact lives. "This account is a demo, until T" is a single-valued fact about the account, so it is an attribute of ACCOUNT (`demo_expires_at`, NULL for every real account), not a new entity. Reusing ACCOUNT means every row a demo creates (profile, contact, list, Journey, sessions, memberships, rules acceptance) is already removed by the existing ON DELETE CASCADE foreign keys: no second deletion path to keep in step.

Table 3: CR-007 changes

Change	Kind	Rule
`account.demo_expires_at`	Attribute (nullable)	CHECK: NULL or later than `created_at`
`idx_account_demo_expires`	Partial index (`WHERE demo_expires_at IS NOT NULL`)	Serves the clean-up `DELETE ... WHERE demo_expires_at < now` without scanning real accounts

Enforced on the server (tested in `worker/demo-login.test.ts`): a demo account has a random username and an unusable random verifier (no password log-in); it reads groups but cannot create a group, report a message or post (the group room refuses its posts); visitors never see each other's data. Real accounts are unchanged (safe default).

2.12 University facts from UGC statistics (read model)

Five facts per institution (students admitted 2023/24, undergraduates 2024, permanent hostel places, Mahapola and Bursary recipients, graduates 2024) come from the UGC report *Sri Lanka University Statistics 2024*, Chapters 2, 3, 4 and 7. They follow the read-model decision used for routes and dates: the extracts (`data/ugc/statistics-2024/*.json`, each number checked twice against the printed table and its totals, and spot-checked by hand) are compiled by `tools/statistics/build-statistics.ts` into `worker/generated/statistics-2024.json`, keyed by institution letter. They are published, versioned reference data that students only read, so they are not stored in D1. Each fact keeps its table, PDF page and URL, and the report's own caveats as flags (2024 provisional; "Information not submitted" shown as no number, never as zero; university totals that include campuses).

2.13 Rooms an account opened (security audit, lead 3)

Group messages live in each room's Durable Object, outside D1's ON DELETE CASCADE. Deleting an account therefore has to tell each room to purge that account's messages, and before 0012 the Worker only knew the groups the account was *still* in, so messages in groups it had left survived the deletion.

Where the fact lives. "Account A opened room G" is a fact about the pair, with no other attributes than when it was first seen, so it is its own relation GROUP_ROOM_SEEN (account_id, group_id, first_at), primary key (account_id, group_id), both foreign keys ON DELETE CASCADE. It is written when the live chat connects (posting is only possible over that socket). Account deletion reads it, unioned with current memberships, **before** deleting the account, then purges each room (at most 45 rooms per request, the free-plan subrequest limit). Migration 0012 seeds it from current memberships so nothing is missed for existing accounts.

2.14 Entity catalogue

Table 4: Entities, their type, identifier and the requirements that justify them

Entity	Type	Identifier	Justified by
STREAM, SUBJECT	Strong	StreamCode; SubjectCode	BR-010 to BR-012
DISTRICT	Strong	DistrictCode	BR-006, BR-007
INSTITUTION	Strong, superclass of UNIVERSITY, CAMPUS, HIGHER_ED_INSTITUTE (disjoint, total)	Letter	BR-015
COURSE	Strong	CourseCode (3 digits)	BR-015
OFFERING	Weak, owners COURSE and INSTITUTION	(CourseCode, Letter); UniCode derived	BR-015
INTAKE_YEAR	Strong	AcademicYear	BR-028
OFFERING_YEAR	Weak, owners OFFERING and INTAKE_YEAR	(UniCode, AcademicYear)	BR-013, BR-014, BR-025, BR-026
SELECTION_GROUP	Weak, owner OFFERING_YEAR	partial key GroupCode	Discovered in the 2025/26 cut-off table
CUTOFF	Weak, owners SELECTION_GROUP and DISTRICT	(group, District)	BR-028, BR-029
RULE_NODE	Strong, recursive	NodeID	BR-020 to BR-022
CITED_FACT	Union category of CUTOFF, OFFERING_YEAR, RULE_NODE, DEADLINE, ROUTE	FactID (surrogate)	BO-2, FR-209, FR-911
SOURCE_DOCUMENT	Strong	SourceID; Sha256 unique	FR-901, FR-902
FACT_REVISION	Weak, owner CITED_FACT	partial key Version	FR-911
EXTRACTION_RUN, CANDIDATE_FACT	Strong	RunID; CandidateID	FR-903 to FR-907
ACCOUNT	Strong, superclass of CURATOR, SENIOR, TEACHER, STUDENT (disjoint, partial)	AccountID	NFR-032, FR-704, FR-706, CR-001
STUDENT	Subclass of ACCOUNT	AccountID (inherited)	CR-001
PASSWORD_LOGIN, RECOVERY_CODE	Weak, owner ACCOUNT, 1:1	AccountID (owner's key); Username unique	CR-001
SESSION	Strong, N:1 to ACCOUNT	TokenHash	CR-001
ACHIEVEMENT	Weak, owner STUDENT	partial key Position	CR-001, BR on special intakes (handbook Section 6)
MISTAKE_REPORT	Strong	ReportID	FR-910
DEADLINE	Strong	DeadlineID	FR-401 to FR-403

Entity	Type	Identifier	Justified by
ROUTE	Strong, superclass of six route groups (disjoint, total)	RouteID	FR-501 to FR-507
PUSH_SUBSCRIPTION	Strong	SubscriptionID	FR-405, FR-406

3 Rule grammar

A requirement is stored as a tree of `rule_node` rows. Evaluation assigns each of the student's three A/L subjects to **at most one** leaf, which captures the handbook's meaning of phrases such as "and the third subject from".

Table 5: Rule node kinds and their meaning

Kind	Meaning	Example (handbook pattern)
ALL	Every child is satisfied, by distinct subjects	Fixed trio (pattern 1)
ANY	At least one child is satisfied	"Combined Mathematics **or** Physics"
AT_LEAST *n*	At least *n* children are satisfied, by distinct subjects	"two other subjects from the list" (pattern 7)
SUBJECT *s* ≥ *g*	Leaf: the student has subject *s* at grade *g* or better, and it is not used elsewhere	"a 'C' grade in Chemistry"
ANY_SUBJECTS *n* ≥ *g*	*n* further subjects of any kind, not used elsewhere	"any other two subjects" (pattern 8)
GRADE_COUNT *n* ≥ *g*	A check on subjects already used: at least *n* of the listed subjects reach grade *g*	Medicine: two 'C' grades (pattern 2)
STREAM_IS *x*	Leaf: the student's stream is *x*	"any three subjects in the Commerce Stream" (pattern 13)
PREDICATE *name*	A named check implemented in code	ARTS_BASKETS: the Arts basket rules (pattern 17)
OL_SUBJECT *s* ≥ *g*	Leaf in the O/L tree: the O/L subject *s* at grade *g* or better	"a Credit Pass in English" (pattern 18)

Example: Medicine (Uni-Code 001A) is ALL(SUBJECT Biology ≥ S, SUBJECT Chemistry ≥ S, SUBJECT Physics ≥ S, GRADE_COUNT 2 ≥ C of {Biology, Chemistry, Physics}). Category splits with separate seats (pattern 14) are modelled as separate selection groups, each with its own rule tree.

4 Mapping the EER model to relations

The relational schema was derived with the standard nine-step algorithm.

Table 6: EER-to-relational mapping steps applied

Step	Construct	Applied to	Result
1	Strong entities	STREAM, SUBJECT, DISTRICT, COURSE, INTAKE_YEAR, RULE_NODE, SOURCE_DOCUMENT, EXTRACTION_RUN, CANDIDATE_FACT, MISTAKE_REPORT, DEADLINE, PUSH_SUBSCRIPTION, ACCOUNT, ROUTE	One table each, primary key from the key attribute
2	Weak entities	OFFERING; OFFERING_YEAR; SELECTION_GROUP; CUTOFF; FACT_REVISION	Owners' keys become part of the primary key: `offering(course_code, institution_letter)`, `cutoff(group_id, district_code)`, `fact_revision(fact_id, version)`. OFFERING_YEAR and SELECTION_GROUP also get surrogate keys to keep later foreign keys short
3	1:1 relationships	CANDIDATE_FACT PUBLISHED_AS CITED_FACT	Foreign key `published_fact_id`
4	1:N relationships	PART_OF, CHILD_OF, TESTS, REQUIRES, READS, PRODUCED, ABOUT, RESOLVES, REVIEWS, FOR_YEAR	Foreign key on the N side; REVIEWS' attributes (decision, note, time) move into `candidate_fact`
5	M:N relationships	OFFERED_IN, CITES, CONCERNS, REMINDS	Relationship tables `stream_subject`, `citation` (with page and quote), `deadline_offering_year`, `push_subscription_deadline`
6	Multi-valued attributes	Medium of OFFERING_YEAR	Table `offering_year_medium`
7	N-ary relationships	None	Not needed
8	Specialisation	INSTITUTION; ACCOUNT	Option 8C, one table with a discriminator (`kind`, `role`): subclasses have almost no own attributes
8	Specialisation	ROUTE	Option 8A: superclass table plus `route_private_degree` and `route_job_exam` for subclasses with several constrained attributes; single-attribute subclasses keep the attribute on `route` with a guard constraint
9	Union category	CITED_FACT	Surrogate key `fact_id` in table `fact`; every member table carries a unique foreign key to it

The student-account constructs (CR-001) were mapped with the same algorithm:

Table 7: Mapping of the student-account model

Step	Construct	Result
2	Weak entities PASSWORD_LOGIN, RECOVERY_CODE (1:1 with the owner)	`password_login`, `recovery_code`, primary key = owner's `account_id` (at most one per account). Kept apart from `account` so staff rows carry no password columns and each sign-in method can be added or removed alone
2	Weak entity ACHIEVEMENT	`student_achievement` with surrogate `achievement_id`; the partial key becomes UNIQUE (`account_id`, `position`)
4	1:N SIGNED_IN	Foreign key `session.account_id`
4	1:N SAT_IN, SAT_FROM and the Z-score	Moved, with the Z-score, into `student_profile` keyed by `account_id`. A separate relation because participation is partial: an account may exist before results are entered, so no NULL-filled columns on `account` (option for partial 1:1 participation)
5	M:N TOOK (Grade, Position)	`student_subject(account_id, position, subject_code, grade)`; at most three per student through CHECK (position 1 to 3), no subject twice through UNIQUE
6	Multi-valued Interest	`student_interest(account_id, interest)`
8	Specialisation STUDENT	Option 8C as before: the `role` discriminator gains `STUDENT`; triggers refuse student rows for any other role

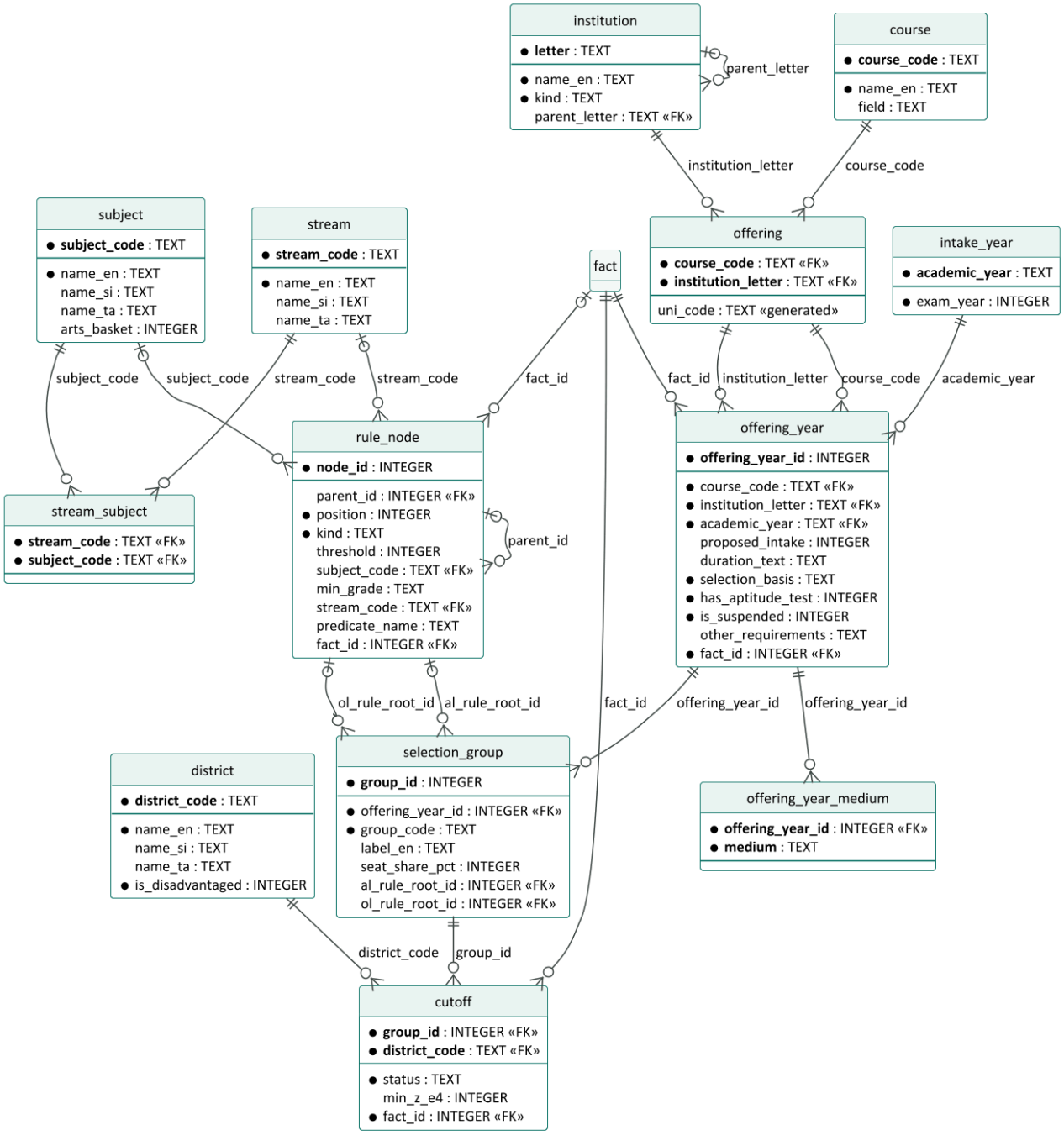


Figure 6: Logical schema, admission data (generated from the migration file)

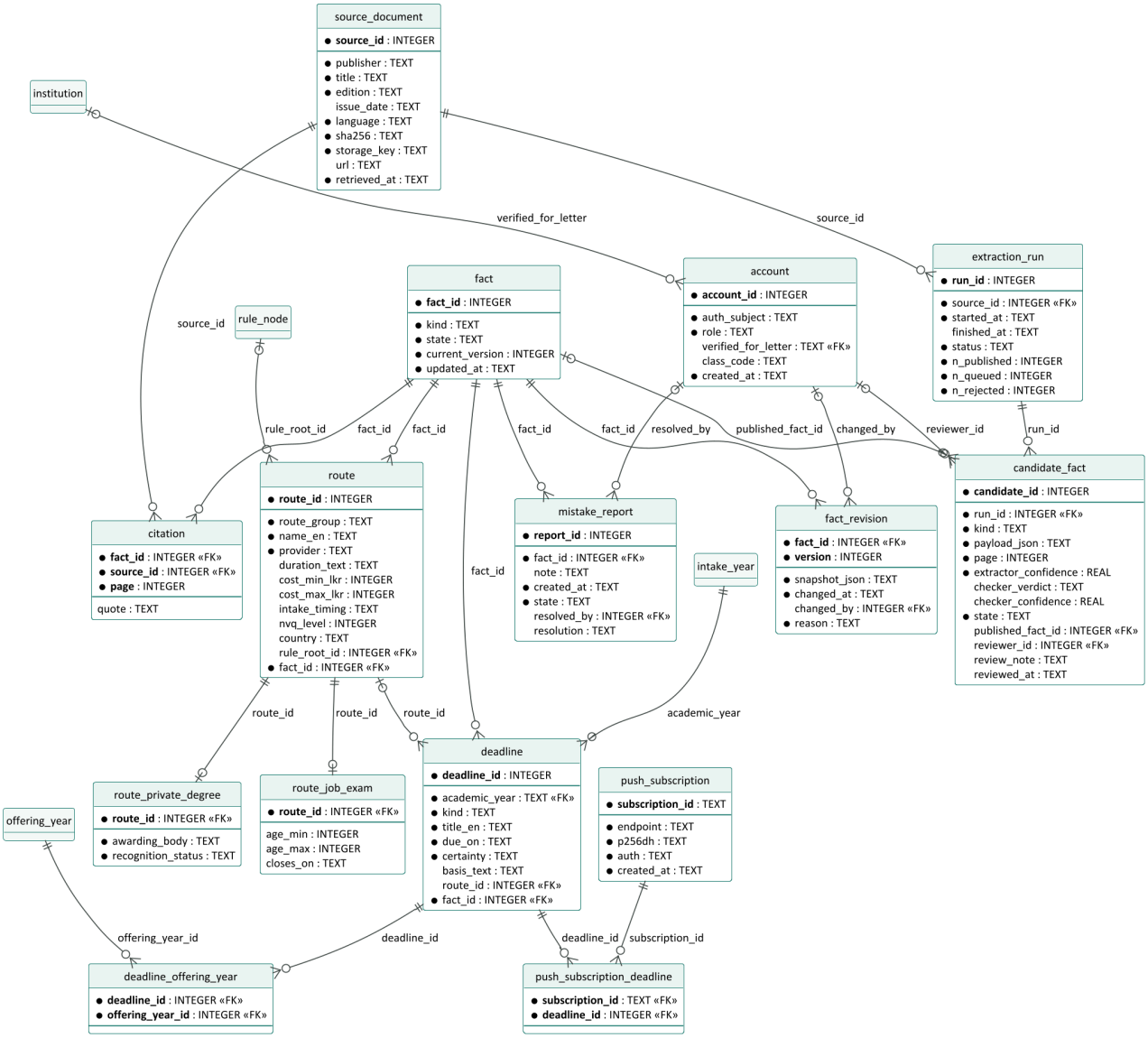


Figure 7: Logical schema, trust, routes and journey data (generated from the migration file)

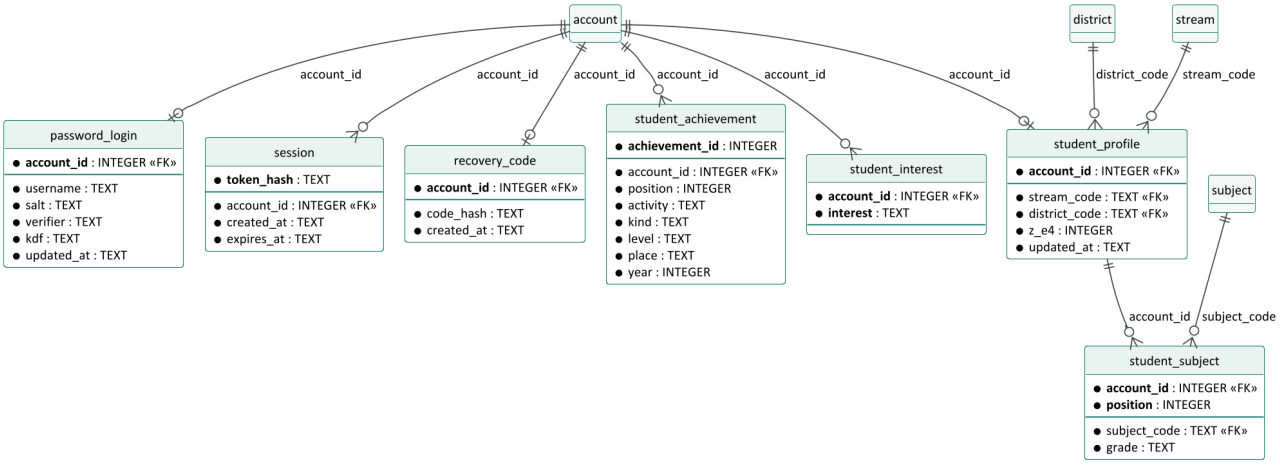


Figure 8: Logical schema, optional student accounts (generated from the migration files)

5 Normalisation

5.1 From the source form to 1NF

The UGC cut-off table, as printed, is a single wide relation: one row per district and one column per offering, with each column header packing several facts into one string, for example "MANAGEMENT STUDIES (TV) - A [Commerce Stream] (University of Vavuniya)", plus markers * (merit only) and # (aptitude test). It violates first normal form twice: a **repeating group** (260 offering columns) and **non-atomic values** (the header). Unpivoting gives one row per (offering column, district), and splitting the header gives atomic attributes:

> R1 (UniCode, CourseCode, CourseName, Letter, InstitutionName, Year, GroupCode, District, IsDisadvantaged, MinZ, Status, ProposedIntake, SelectionBasis, HasAptitudeTest, Medium)

5.2 Functional and multi-valued dependencies

Table 8: Dependencies identified in R1

#	Dependency	Source of the rule
FD1	UniCode → CourseCode, Letter (and the reverse: CourseCode, Letter → UniCode)	BR-015
FD2	CourseCode → CourseName	Handbook course list
FD3	Letter → InstitutionName	Handbook Uni-Code list
FD4	District → IsDisadvantaged	BR-007
FD5	UniCode, Year → ProposedIntake, SelectionBasis, HasAptitudeTest	Handbook, per intake
FD6	UniCode, Year, GroupCode, District → MinZ, Status	Cut-off table
MVD1	UniCode, Year ↠ Medium	Medium is independent of every other attribute

The candidate key of R1 is (UniCode, Year, GroupCode, District, Medium).

5.3 2NF, 3NF and BCNF

- **2NF** removes attributes that depend on part of the key: FD5's attributes move to OFFERING_YEAR(UniCode, Year, ...); FD4's to DISTRICT; FD1 to OFFERING. CUTOFF keeps only MinZ and Status, which need the whole key.
- **3NF** removes transitive dependencies: UniCode → CourseCode → CourseName puts CourseName in COURSE; UniCode → Letter → InstitutionName puts the name in INSTITUTION.
- **BCNF**: in every resulting relation, every determinant is a candidate key. The only relation with two candidate keys, OFFERING, has (CourseCode, Letter) and the derived UniCode, both keys, so it is in BCNF.
- **4NF**: MVD1 is non-trivial, so Medium moves to its own relation, `offering_year_medium`.

The same analysis applied to the provenance data separates `citation` (fact, source, page) from `fact`, because a fact may cite several pages and a page supports many facts.

The student-account relations are in BCNF by construction. In each, the only determinants are the keys: `password_login` has two candidate keys, `account_id` and `username`, and every other attribute depends on either. `student_subject` has the candidate keys (`account_id`, `position`) and (`account_id`, `subject_code`), with Grade depending on both. A student's interests are independent of their achievements (MVD AccountID ↠ Interest), so they are kept in separate relations (4NF).

5.4 Deliberate departures

Table 9: Controlled denormalisations and why they are safe

Departure	Reason	How consistency is kept
`uni_code` stored alongside its parts in `offering`	Lookup by the code students know	Generated column, computed by the database; cannot disagree
Surrogate keys on `offering_year` and `selection_group`	Short foreign keys in the 13,000-row `cutoff` table	Natural keys kept as UNIQUE constraints
`journey_notice.route_id` without a foreign key (CR-006)	The route catalogue is curated read-model data compiled into the Worker; the D1 `route` table is not populated	CHECK on the id's shape in D1; the Worker accepts only ids in the catalogue (tested)
Avatar copied onto each group message (CR-006)	A message shows the picture the author had when posting, like the author name; no join across storage	The copy is taken from the account at connection time; deleting the account deletes its messages
Read model for the student path (ZP-DOC-06)	D1 free plan allows 5 million row reads a day; a district's full eligibility view reads about 3,000 rows, which would cap ZedPath near 1,600 students a day	Built from D1 by a single job after each data change, versioned, and never edited directly

6 Physical design for Cloudflare D1

6.1 Conventions

- **STRICT tables**: every value must match its declared type.
- **Exact Z-scores**: stored as integers in ten-thousandths (`min_z_e4`; 1.4821 is stored as 14821), so comparisons are exact (NFR-051). The valid range −4.0000 to +4.0000 is a CHECK constraint (BR-005).
- **Enumerations** (streams, statuses, kinds, grades) are CHECK constraints, so invalid states cannot be stored.
- **Cross-column rules** are CHECK constraints: a CAMPUS must have a parent university; a VALUE cut-off must have a number and an NQC cut-off must not; an auto-published candidate must have two-model agreement at 0.90 or more (FR-905); an estimated deadline must state its basis (FR-402).
- **Foreign keys** are enforced by D1 by default (equivalent to `PRAGMA foreign_keys = ON`).
- **Personal data**: without an account, the only student-linked table is `push_subscription`, holding a browser push endpoint and keys that the student opted in to share (NFR-030). With an optional account (CR-001), the student's results, achievements and interests are kept under a username they choose; there are still no names, contact details or telephone numbers. `DELETE FROM account` cascades to every row of that student.
- **Secrets are stored only as hashes** (CR-001): `password_login.verifier` = SHA-256 of a key that the device derives with PBKDF2-HMAC-SHA256 at 600,000 rounds (the password never reaches the server); `session.token_hash` = SHA-256 of the random cookie token; `recovery_code.code_hash` = SHA-256 of an 80-bit random code. A copy of the tables therefore cannot be replayed to log in, and a guessed password costs an attacker 600,000 rounds per guess. Verifying a login costs the Worker one SHA-256, well inside the free plan's 10 ms CPU limit.

6.2 Indexes

Table 10: Secondary indexes and the query each serves

Index	Serves
`idx_account_demo_expires` (partial, CR-007)	Deleting expired demo accounts on each demo tap
`idx_offering_uni_code` (unique)	Look up an offering by Uni-Code
`idx_cutoff_district`	All cut-offs for one district: the core of eligibility and banding
`idx_offering_year_year`	All offerings of an intake year
`idx_rule_node_parent`	Loading rule trees in order
`idx_deadline_year_due`	Journey timeline by date
`idx_citation_source`	All facts citing a source (re-extraction, audits)
`idx_candidate_queue` (partial, QUEUED only)	Curator review queue
`idx_report_open` (partial, OPEN only)	Open mistake reports
`idx_session_account`	Log out on every device; clearing expired sessions at log-in
`idx_student_profile_stream`, `idx_student_profile_district`	Foreign-key checks when a stream or district row changes; counts for reminders by district
`idx_student_subject_subject`	Foreign-key checks on `subject`

6.3 Capacity against the free plan

Table 11: Data volume per intake year and free-plan headroom

Measure	Value	Free-plan limit
Cut-off rows per intake year	about 6,500	—
Database size with two intake years loaded	2.3 MB	500 MB per database
Projected size with five years and all routes	under 10 MB	500 MB per database
Rows read to rebuild the read model once	about 15,000	5 million per day
Rows written to load one intake year	about 13,500 (with facts and citations)	100,000 per day

7 Validation against real data

The design was tested by loading the official data, not sample data. `tools/seed/validate-schema.ts` applies the migration to an in-memory SQLite database (the engine D1 is built on), loads the UGC 2025/26 data and the 2024/25 cut-offs printed in the handbook, runs integrity checks, and then attempts 24 deliberately invalid inserts.

Table 12: Schema validation results (5 October 2026)

Check	Result	Expected
Uni-Codes loaded	255	255
Course codes / institutions	121 / 20	121 / 20
2025/26 proposed intake across Uni-Codes (plus 900 Arts additional intake = 42,937)	42,037	42,037
2025/26 cut-off columns matched exactly to a Uni-Code	260	260
2025/26 cut-off cells loaded / of which NQC	6,500 / 1,026	6,500 / 1,026
Uni-Codes split into more than one selection group	5	5
Uni-Codes without cut-offs	0	0
Merit-only offerings with differing district values	0	0
2024/25 cut-off cells loaded	6,425	—
Facts without a citation	0	0
Foreign-key violations	0	0
Invalid inserts rejected by constraints	24 of 24	24 of 24

The student-account tables (CR-001) are validated by `worker/account.test.ts`, which applies all three migrations to an in-memory SQLite database with foreign keys on, exactly as D1 enforces them, and drives the real API.

Table 13: Student-account validation results (5 October 2026)

Check	Result
Schema after migrations 0001 to 0003	35 tables, 190 columns, 52 foreign keys, 12 indexes, 3 triggers
Reference rows loaded by 0002 (streams / districts / subjects)	6 / 25 / 59
Invalid inserts rejected (unknown role, bad username, wrong-length hash, unknown KDF, Z-score out of range, unknown stream, district or subject, fourth subject, duplicate subject, grade D, blank or unknown achievement fields, unknown interest, expiry before creation, student rows on a staff account)	17 of 17
Deleting one account removes exactly its rows from all 8 account tables and nothing of another account	Pass
A request can read or change only its own session's account, whatever the request body says	Pass
A save that fails validation changes nothing (atomic batch)	Pass

Check	Result
Bugs planted on purpose (body-supplied account id, missing cascade, missing cross-site check, key not compared) caught by a failing test	4 of 4
Same migrations applied to the local D1 runtime (`wrangler d1 migrations apply --local`)	3 of 3 applied

Matching the cut-off table to Uni-Codes surfaced spelling differences between the two UGC documents ("BIO.SC" and "BIOLOGICAL SC.", "AGRI BUSINESS" and "AGRIBUSINESS", "BIO RESOURCES" and "BIORESOURCES") and one misprinted university name. The loader normalises these explicitly; no column is matched by guesswork.

Appendix A: Traceability: requirements to tables

Table 14: Requirements and the tables that realise them

Requirement	Tables
BR-005, NFR-051 (exact Z-scores)	`cutoff.min_z_e4` with range CHECK
BR-006, BR-007 (districts)	`district`
BR-010 to BR-012 (streams, subjects)	`stream`, `subject`, `stream_subject`
BR-013, BR-014 (quotas, merit-only)	`offering_year.selection_basis`
BR-015 (Uni-Codes)	`course`, `institution`, `offering`
BR-020 to BR-024 (requirements)	`rule_node`, `selection_group.al_rule_root_id`, `ol_rule_root_id`, `offering_year.other_requirements`
BR-025, BR-026 (aptitude, suspended)	`offering_year.has_aptitude_test`, `is_suspended`
BR-028, BR-029 (cut-offs, NQC)	`selection_group`, `cutoff`
FR-209, BO-2, NFR-050 (every fact sourced)	`fact`, `citation`, `source_document`
FR-401 to FR-406 (journey, reminders)	`deadline`, `deadline_offering_year`, `push_subscription`, `push_subscription_deadline`
FR-501 to FR-507 (other routes)	`route`, `route_private_degree`, `route_job_exam`
FR-901 to FR-911 (data and trust)	`source_document`, `extraction_run`, `candidate_fact`, `fact_revision`, `mistake_report`, `account`
CR-001 (optional student account, cross-device profile)	`account` (role STUDENT), `password_login`, `session`, `recovery_code`, `student_profile`, `student_subject`, `student_achievement`, `student_interest`
CR-002 (Groups; Journey and Alerts as read-model data)	`course`, `institution` (0005), `course_group`, `group_member`, `senior_invite`, `message_report`, `account_block` (0006)
CR-003 (compulsory contact details, consent)	`student_contact` (0007)
CR-004 (Ask answer feedback)	`answer_feedback` (0008)
CR-006 (rules once per account, avatars, Journey on the account)	`group_rules_acceptance`, `account.avatar`, `journey_course` (to `offering`), `journey_notice`, `journey_step` (0010)
CR-007 (one-tap demo, a private copy per visitor)	`account.demo_expires_at`, `idx_account_demo_expires` (0011)
Security audit run-1, lead 3 (deletion purges every room)	`group_room_seen` (0012)

Appendix B: Physical schema (migration 0001)

Reproduced verbatim from `migrations/0001_initial_schema.sql`.

```
-- ZedPath D1 schema, migration 0001.
-- Designed in ZP-DOC-05 Data Design (conceptual EER -> relational mapping -> normalisation to
BCNF/4NF).
-- Conventions: STRICT tables; snake_case; exact Z-scores as INTEGER ten-thousandths (z_e4 = Z x
10000,
-- e.g. 1.4821 -> 14821) so comparisons are exact (NFR-051); timestamps as ISO 8601 UTC text;
-- every published fact has a row in `fact` and at least one `citation` (BO-2, NFR-050).
-- D1 enforces foreign keys by default (equivalent to PRAGMA foreign_keys = ON).

-- ------------------------------------------------------------------ provenance (needed first: other
tables reference fact)

CREATE TABLE fact (
  fact_id          INTEGER PRIMARY KEY,
  kind             TEXT    NOT NULL CHECK (kind IN
('CUTOFF','OFFERING_YEAR','RULE','DEADLINE','ROUTE')),
  state            TEXT    NOT NULL DEFAULT 'PUBLISHED' CHECK (state IN ('PUBLISHED','WITHDRAWN')),
  current_version  INTEGER NOT NULL DEFAULT 1 CHECK (current_version >= 1),
  updated_at       TEXT    NOT NULL CHECK (updated_at GLOB '[0-9][0-9][0-9][0-9]-[0-9][0-9]-[0-9][0-
9]T*Z')
) STRICT;

CREATE TABLE source_document (
  source_id     INTEGER PRIMARY KEY,
  publisher     TEXT NOT NULL,
  title         TEXT NOT NULL,
  edition       TEXT NOT NULL,                        -- e.g. 'Academic year 2025/2026'
  issue_date    TEXT CHECK (issue_date IS NULL OR issue_date GLOB '[0-9][0-9][0-9][0-9]-[0-9][0-9]-
[0-9][0-9]'),
  language      TEXT NOT NULL CHECK (language IN ('EN','SI','TA')),
  sha256        TEXT NOT NULL UNIQUE CHECK (length(sha256) = 64 AND sha256 NOT GLOB '*[^0-9a-f]*'),
  storage_key   TEXT NOT NULL UNIQUE,                 -- R2 object key; the file itself is never
redistributed
  url           TEXT,
  retrieved_at  TEXT NOT NULL
) STRICT;

CREATE TABLE citation (
  fact_id    INTEGER NOT NULL REFERENCES fact(fact_id) ON DELETE CASCADE,
  source_id  INTEGER NOT NULL REFERENCES source_document(source_id),
  page       INTEGER NOT NULL CHECK (page >= 1),      -- printed page number in the source
  quote      TEXT,                                    -- short verbatim quote, where useful
  PRIMARY KEY (fact_id, source_id, page)
) STRICT;
CREATE INDEX idx_citation_source ON citation(source_id);

CREATE TABLE account (
  account_id           INTEGER PRIMARY KEY,
  auth_subject         TEXT NOT NULL UNIQUE,         -- identity-provider subject; no passwords
stored
  role                 TEXT NOT NULL CHECK (role IN ('CURATOR','SENIOR','TEACHER')),
  verified_for_letter  TEXT REFERENCES institution(letter),
  class_code           TEXT UNIQUE,
  created_at           TEXT NOT NULL,
  CHECK ((role = 'SENIOR')  = (verified_for_letter IS NOT NULL)),
  CHECK ((role = 'TEACHER') = (class_code IS NOT NULL))
) STRICT;

CREATE TABLE fact_revision (
  fact_id        INTEGER NOT NULL REFERENCES fact(fact_id) ON DELETE CASCADE,
  version        INTEGER NOT NULL CHECK (version >= 1),
  snapshot_json  TEXT    NOT NULL CHECK (json_valid(snapshot_json)),
  changed_at     TEXT    NOT NULL,
  changed_by     INTEGER REFERENCES account(account_id),
```

```
  reason         TEXT    NOT NULL,
  PRIMARY KEY (fact_id, version)
) STRICT;

-- ---------------------------------------------------------------- reference data

CREATE TABLE stream (
  stream_code  TEXT PRIMARY KEY CHECK (stream_code IN ('ARTS','COMMERCE','BIO','PHYS','ET','BST')),
  name_en      TEXT NOT NULL,
  name_si      TEXT,
  name_ta      TEXT
) STRICT;

CREATE TABLE subject (
  subject_code  TEXT PRIMARY KEY CHECK (length(subject_code) BETWEEN 2 AND 12 AND subject_code NOT
GLOB '*[^A-Z0-9_]*'),
  name_en       TEXT NOT NULL UNIQUE,
  name_si       TEXT,
  name_ta       TEXT,
  arts_basket   INTEGER CHECK (arts_basket IS NULL OR arts_basket BETWEEN 1 AND 4)
) STRICT;

CREATE TABLE stream_subject (
  stream_code   TEXT NOT NULL REFERENCES stream(stream_code),
  subject_code  TEXT NOT NULL REFERENCES subject(subject_code),
  PRIMARY KEY (stream_code, subject_code)
) STRICT, WITHOUT ROWID;

CREATE TABLE district (
  district_code     TEXT PRIMARY KEY CHECK (length(district_code) = 3 AND district_code NOT GLOB
'*[^A-Z]*'),
  name_en           TEXT NOT NULL UNIQUE,
  name_si           TEXT,
  name_ta           TEXT,
  is_disadvantaged  INTEGER NOT NULL CHECK (is_disadvantaged IN (0,1))   -- BR-007
) STRICT;

-- Specialisation UNIVERSITY / CAMPUS / HIGHER_ED_INSTITUTE mapped to one table with a discriminator
-- (Elmasri and Navathe option 8C): the subclasses have almost no own attributes.
CREATE TABLE institution (
  letter         TEXT PRIMARY KEY CHECK (length(letter) = 1 AND letter GLOB '[A-Z]'),
  name_en        TEXT NOT NULL UNIQUE,
  kind           TEXT NOT NULL CHECK (kind IN ('UNIVERSITY','CAMPUS','HEI')),
  parent_letter  TEXT REFERENCES institution(letter),
  CHECK ((kind = 'CAMPUS') = (parent_letter IS NOT NULL))
) STRICT;

CREATE TABLE course (
  course_code  TEXT PRIMARY KEY CHECK (course_code GLOB '[0-9][0-9][0-9]'),
  name_en      TEXT NOT NULL,
  field        TEXT
) STRICT;

-- Weak entity OFFERING, identified by COURSE and INSTITUTION; the Uni-Code is derived (BR-015).
CREATE TABLE offering (
  course_code         TEXT NOT NULL REFERENCES course(course_code),
  institution_letter  TEXT NOT NULL REFERENCES institution(letter),
  uni_code            TEXT GENERATED ALWAYS AS (course_code || institution_letter) STORED,
  PRIMARY KEY (course_code, institution_letter)
) STRICT;
CREATE UNIQUE INDEX idx_offering_uni_code ON offering(uni_code);

CREATE TABLE intake_year (
  academic_year  TEXT PRIMARY KEY CHECK (academic_year GLOB '[0-9][0-9][0-9][0-9]/[0-9][0-9][0-9][0-
9]'),
  exam_year      INTEGER NOT NULL,
  CHECK (exam_year = CAST(substr(academic_year, 1, 4) AS INTEGER)),
  CHECK (CAST(substr(academic_year, 6, 4) AS INTEGER) = exam_year + 1)
) STRICT;
```

```
-- ------------------------------------------------------------ requirement rules (recursive
rule tree)
-- Kinds: ALL / ANY (boolean), AT_LEAST n of the children, SUBJECT (leaf: subject at min_grade),
-- ANY_SUBJECTS n (n further subjects of any kind), GRADE_COUNT (at least n of the child subjects at
min_grade,
-- a check on subjects already used), STREAM_IS (leaf), PREDICATE (named check implemented in code,
e.g. ARTS_BASKETS),
-- OL_SUBJECT (leaf: G.C.E. O/L subject at min_grade). Semantics in ZP-DOC-05 Section "Rule
grammar".
CREATE TABLE rule_node (
  node_id         INTEGER PRIMARY KEY,
  parent_id       INTEGER REFERENCES rule_node(node_id) ON DELETE CASCADE,
  position        INTEGER NOT NULL DEFAULT 0 CHECK (position >= 0),
  kind            TEXT    NOT NULL CHECK (kind IN
('ALL','ANY','AT_LEAST','SUBJECT','ANY_SUBJECTS','GRADE_COUNT',
                                                'STREAM_IS','PREDICATE','OL_SUBJECT')),
  threshold       INTEGER CHECK (threshold IS NULL OR threshold BETWEEN 1 AND 3),
  subject_code    TEXT REFERENCES subject(subject_code),
  min_grade       TEXT CHECK (min_grade IS NULL OR min_grade IN ('A','B','C','S')),
  stream_code     TEXT REFERENCES stream(stream_code),
  predicate_name  TEXT CHECK (predicate_name IS NULL OR predicate_name IN ('ARTS_BASKETS')),
  fact_id         INTEGER UNIQUE REFERENCES fact(fact_id),   -- set on root nodes only: the rule is
the cited fact
  CHECK (kind NOT IN ('SUBJECT','OL_SUBJECT') OR (subject_code IS NOT NULL AND min_grade IS NOT
NULL)),
  CHECK (kind NOT IN ('AT_LEAST','ANY_SUBJECTS','GRADE_COUNT') OR threshold IS NOT NULL),
  CHECK (kind <> 'GRADE_COUNT' OR min_grade IS NOT NULL),
  CHECK (kind <> 'ANY_SUBJECTS' OR min_grade IS NOT NULL),
  CHECK ((kind = 'STREAM_IS') = (stream_code IS NOT NULL)),
  CHECK ((kind = 'PREDICATE') = (predicate_name IS NOT NULL)),
  CHECK (subject_code IS NULL OR kind IN ('SUBJECT','OL_SUBJECT')),
  CHECK (fact_id IS NULL OR parent_id IS NULL)
) STRICT;
CREATE INDEX idx_rule_node_parent ON rule_node(parent_id, position);

-- ------------------------------------------------------------ offerings per year, selection
groups, cut-offs

CREATE TABLE offering_year (
  offering_year_id       INTEGER PRIMARY KEY,
  course_code            TEXT NOT NULL,
  institution_letter     TEXT NOT NULL,
  academic_year          TEXT NOT NULL REFERENCES intake_year(academic_year),
  proposed_intake        INTEGER CHECK (proposed_intake IS NULL OR proposed_intake >= 0),
  duration_text          TEXT,
  selection_basis        TEXT NOT NULL CHECK (selection_basis IN ('QUOTA','MERIT_ONLY')),   -- BR-
013, BR-014
  has_aptitude_test      INTEGER NOT NULL CHECK (has_aptitude_test IN (0,1)),              -- BR-025
  is_suspended           INTEGER NOT NULL DEFAULT 0 CHECK (is_suspended IN (0,1)),         -- BR-026
  other_requirements     TEXT,                                                             -- BR-023
(not evaluated)
  fact_id                INTEGER NOT NULL UNIQUE REFERENCES fact(fact_id),
  FOREIGN KEY (course_code, institution_letter) REFERENCES offering(course_code,
institution_letter),
  UNIQUE (course_code, institution_letter, academic_year)
) STRICT;
CREATE INDEX idx_offering_year_year ON offering_year(academic_year);

-- Multi-valued attribute Medium (4NF: independent of every other offering-year attribute).
CREATE TABLE offering_year_medium (
  offering_year_id  INTEGER NOT NULL REFERENCES offering_year(offering_year_id) ON DELETE CASCADE,
  medium            TEXT    NOT NULL CHECK (medium IN ('SI','TA','EN')),
  PRIMARY KEY (offering_year_id, medium)
) STRICT, WITHOUT ROWID;

-- Weak entity SELECTION_GROUP: most offerings have one group 'ALL'; a few split their seats by
stream or category
-- (e.g. Food Business Management: Biological/Physical Science vs Commerce), each with its own rule
and cut-offs.
```

```
CREATE TABLE selection_group (
  group_id          INTEGER PRIMARY KEY,
  offering_year_id  INTEGER NOT NULL REFERENCES offering_year(offering_year_id) ON DELETE CASCADE,
  group_code        TEXT    NOT NULL DEFAULT 'ALL' CHECK (length(group_code) BETWEEN 1 AND 16),
  label_en          TEXT,
  seat_share_pct    INTEGER CHECK (seat_share_pct IS NULL OR seat_share_pct BETWEEN 1 AND 100),
  al_rule_root_id   INTEGER REFERENCES rule_node(node_id),
  ol_rule_root_id   INTEGER REFERENCES rule_node(node_id),
  UNIQUE (offering_year_id, group_code)
) STRICT;

-- Weak entity CUTOFF, identified by SELECTION_GROUP and DISTRICT (BR-028, BR-029).
CREATE TABLE cutoff (
  group_id       INTEGER NOT NULL REFERENCES selection_group(group_id) ON DELETE CASCADE,
  district_code  TEXT    NOT NULL REFERENCES district(district_code),
  status         TEXT    NOT NULL CHECK (status IN ('VALUE','NQC')),
  min_z_e4       INTEGER CHECK (min_z_e4 IS NULL OR min_z_e4 BETWEEN -40000 AND 40000),
  fact_id        INTEGER NOT NULL UNIQUE REFERENCES fact(fact_id),
  CHECK ((status = 'VALUE') = (min_z_e4 IS NOT NULL)),
  PRIMARY KEY (group_id, district_code)
) STRICT;
CREATE INDEX idx_cutoff_district ON cutoff(district_code, group_id);

-- ------------------------------------------------------------------ other routes (specialisation
ROUTE)
-- Superclass table + subclass tables for the subclasses with their own constrained attributes
-- (option 8A); single-attribute subclasses keep that attribute on the superclass with a guard
CHECK.

CREATE TABLE route (
  route_id        INTEGER PRIMARY KEY,
  route_group     TEXT NOT NULL CHECK (route_group IN
('PRIVATE_DEGREE','DIPLOMA','PROFESSIONAL','VOCATIONAL',
                                                        'JOB_EXAM','SCHOLARSHIP_ABROAD')),
  name_en         TEXT NOT NULL,
  provider        TEXT NOT NULL,
  duration_text   TEXT,
  cost_min_lkr    INTEGER CHECK (cost_min_lkr IS NULL OR cost_min_lkr >= 0),
  cost_max_lkr    INTEGER CHECK (cost_max_lkr IS NULL OR cost_max_lkr >= 0),
  intake_timing   TEXT,
  nvq_level       INTEGER CHECK (nvq_level IS NULL OR (route_group = 'VOCATIONAL' AND nvq_level
BETWEEN 1 AND 7)),
  country         TEXT    CHECK (country IS NULL OR route_group = 'SCHOLARSHIP_ABROAD'),
  rule_root_id    INTEGER REFERENCES rule_node(node_id),
  fact_id         INTEGER NOT NULL UNIQUE REFERENCES fact(fact_id),
  CHECK (cost_min_lkr IS NULL OR cost_max_lkr IS NULL OR cost_min_lkr <= cost_max_lkr)
) STRICT;

CREATE TABLE route_private_degree (
  route_id            INTEGER PRIMARY KEY REFERENCES route(route_id) ON DELETE CASCADE,
  awarding_body       TEXT NOT NULL,
  recognition_status  TEXT NOT NULL CHECK (recognition_status IN ('LISTED','NOT_LISTED','UNKNOWN'))
) STRICT;

CREATE TABLE route_job_exam (
  route_id   INTEGER PRIMARY KEY REFERENCES route(route_id) ON DELETE CASCADE,
  age_min    INTEGER CHECK (age_min IS NULL OR age_min BETWEEN 14 AND 60),
  age_max    INTEGER CHECK (age_max IS NULL OR age_max BETWEEN 14 AND 60),
  closes_on  TEXT CHECK (closes_on IS NULL OR closes_on GLOB '[0-9][0-9][0-9][0-9]-[0-9][0-9]-[0-
9][0-9]'),
  CHECK (age_min IS NULL OR age_max IS NULL OR age_min <= age_max)
) STRICT;

-- ------------------------------------------------------------------ journey: deadlines and reminders

CREATE TABLE deadline (
  deadline_id    INTEGER PRIMARY KEY,
  academic_year  TEXT NOT NULL REFERENCES intake_year(academic_year),
  kind           TEXT NOT NULL CHECK (kind IN
('UGC_APPLICATION_OPEN','UGC_APPLICATION_CLOSE','PREFERENCE_CHANGE_CLOSE',
```

```
'APTITUDE_APPLICATION','APTITUDE_TEST','CUTOFF_RELEASE','APPEAL_CLOSE',

'REGISTRATION_CLOSE','ROUTE_INTAKE','JOB_EXAM_CLOSE','OTHER')),
  title_en       TEXT NOT NULL,
  due_on         TEXT NOT NULL CHECK (due_on GLOB '[0-9][0-9][0-9][0-9]-[0-9][0-9]-[0-9][0-9]'),
  certainty      TEXT NOT NULL CHECK (certainty IN ('CONFIRMED','ESTIMATED')),                -- BR
/ FR-402
  basis_text     TEXT,
  route_id       INTEGER REFERENCES route(route_id),
  fact_id        INTEGER NOT NULL UNIQUE REFERENCES fact(fact_id),
  CHECK (certainty = 'CONFIRMED' OR basis_text IS NOT NULL)
) STRICT;
CREATE INDEX idx_deadline_year_due ON deadline(academic_year, due_on);

CREATE TABLE deadline_offering_year (
  deadline_id       INTEGER NOT NULL REFERENCES deadline(deadline_id) ON DELETE CASCADE,
  offering_year_id  INTEGER NOT NULL REFERENCES offering_year(offering_year_id) ON DELETE CASCADE,
  PRIMARY KEY (deadline_id, offering_year_id)
) STRICT, WITHOUT ROWID;

-- Only data a student explicitly opted in to (NFR-030): a push endpoint, never a name or contact
detail.
CREATE TABLE push_subscription (
  subscription_id  TEXT PRIMARY KEY CHECK (length(subscription_id) = 32),
  endpoint         TEXT NOT NULL UNIQUE CHECK (endpoint GLOB 'https://*'),
  p256dh           TEXT NOT NULL,
  auth             TEXT NOT NULL,
  created_at       TEXT NOT NULL
) STRICT;

CREATE TABLE push_subscription_deadline (
  subscription_id  TEXT    NOT NULL REFERENCES push_subscription(subscription_id) ON DELETE CASCADE,
  deadline_id      INTEGER NOT NULL REFERENCES deadline(deadline_id) ON DELETE CASCADE,
  PRIMARY KEY (subscription_id, deadline_id)
) STRICT, WITHOUT ROWID;

-- ------------------------------------------------------------------ extraction, review and reports

CREATE TABLE extraction_run (
  run_id       INTEGER PRIMARY KEY,
  source_id    INTEGER NOT NULL REFERENCES source_document(source_id),
  started_at   TEXT NOT NULL,
  finished_at  TEXT,
  status       TEXT NOT NULL CHECK (status IN ('RUNNING','SUCCEEDED','FAILED','PAUSED')),
  n_published  INTEGER NOT NULL DEFAULT 0 CHECK (n_published >= 0),
  n_queued     INTEGER NOT NULL DEFAULT 0 CHECK (n_queued >= 0),
  n_rejected   INTEGER NOT NULL DEFAULT 0 CHECK (n_rejected >= 0),
  CHECK ((status = 'RUNNING') = (finished_at IS NULL))
) STRICT;

-- REVIEWS (1:N, with attributes) is mapped into candidate_fact with the foreign-key approach
(mapping step 4).
CREATE TABLE candidate_fact (
  candidate_id          INTEGER PRIMARY KEY,
  run_id                INTEGER NOT NULL REFERENCES extraction_run(run_id) ON DELETE CASCADE,
  kind                  TEXT    NOT NULL CHECK (kind IN
('CUTOFF','OFFERING_YEAR','RULE','DEADLINE','ROUTE')),
  payload_json          TEXT    NOT NULL CHECK (json_valid(payload_json)),
  page                  INTEGER NOT NULL CHECK (page >= 1),
  extractor_confidence  REAL    NOT NULL CHECK (extractor_confidence BETWEEN 0 AND 1),
  checker_verdict       TEXT    CHECK (checker_verdict IS NULL OR checker_verdict IN
('AGREE','DISAGREE')),
  checker_confidence    REAL    CHECK (checker_confidence IS NULL OR checker_confidence BETWEEN 0
AND 1),
  state                 TEXT    NOT NULL CHECK (state IN
('AUTO_PUBLISHED','QUEUED','APPROVED','CORRECTED','REJECTED')),
  published_fact_id     INTEGER REFERENCES fact(fact_id),
  reviewer_id           INTEGER REFERENCES account(account_id),
  review_note           TEXT,
```

```
  reviewed_at           TEXT,
  CHECK ((state IN ('AUTO_PUBLISHED','APPROVED','CORRECTED')) = (published_fact_id IS NOT NULL)),
  CHECK ((state IN ('APPROVED','CORRECTED','REJECTED')) = (reviewer_id IS NOT NULL AND reviewed_at
IS NOT NULL)),
  CHECK (state <> 'AUTO_PUBLISHED' OR (checker_verdict = 'AGREE' AND extractor_confidence >= 0.90
                                       AND checker_confidence >= 0.90))                    -- FR-
905
) STRICT;
CREATE INDEX idx_candidate_queue ON candidate_fact(state) WHERE state = 'QUEUED';

CREATE TABLE mistake_report (
  report_id    INTEGER PRIMARY KEY,
  fact_id      INTEGER NOT NULL REFERENCES fact(fact_id),
  note         TEXT CHECK (note IS NULL OR length(note) <= 1000),
  created_at   TEXT NOT NULL,
  state        TEXT NOT NULL DEFAULT 'OPEN' CHECK (state IN ('OPEN','FIXED','CLOSED')),
  resolved_by  INTEGER REFERENCES account(account_id),
  resolution   TEXT,
  CHECK ((state = 'OPEN') = (resolved_by IS NULL))
) STRICT;
CREATE INDEX idx_report_open ON mistake_report(state) WHERE state = 'OPEN';
```

Appendix C: Student accounts (migration 0003)

Reproduced verbatim from `migrations/0003_student_accounts.sql`. Migration 0002 inserts the reference data (6 streams, 25 districts, 59 subjects) and is generated by `tools/seed/reference-sql.ts` from the compiled rulebook.

```
-- ZedPath D1 migration 0003: student accounts (CR-001, approved by Praveen 2026-10-05 "oo
accounts").
-- Design: ZP-DOC-05 Section "Student accounts". One identity table (account) for every role; each
way of signing in
-- is its own table hanging off it (password_login now, an identity provider such as Google later).
-- Privacy (NFR-030): a username only. No name, email or phone is ever stored. Deleting an account
deletes every row
-- below it (ON DELETE CASCADE), enforced by D1, which always enforces foreign keys.

-- account: add the STUDENT role. SQLite cannot change a CHECK in place, so the table is rebuilt. It
holds no rows
-- yet in any environment (no staff accounts exist), so the copy is a no-op kept for correctness.
CREATE TABLE account_v2 (
  account_id           INTEGER PRIMARY KEY,
  auth_subject         TEXT NOT NULL UNIQUE,         -- opaque: 'local:<32 hex>' for password
logins, provider subject later
  role                 TEXT NOT NULL CHECK (role IN ('CURATOR','SENIOR','TEACHER','STUDENT')),
  verified_for_letter  TEXT REFERENCES institution(letter),
  class_code           TEXT UNIQUE,
  created_at           TEXT NOT NULL,
  CHECK ((role = 'SENIOR')  = (verified_for_letter IS NOT NULL)),
  CHECK ((role = 'TEACHER') = (class_code IS NOT NULL))
) STRICT;
INSERT INTO account_v2 SELECT account_id, auth_subject, role, verified_for_letter, class_code,
created_at FROM account;
DROP TABLE account;
ALTER TABLE account_v2 RENAME TO account;

-- Username + password. The phone derives key = PBKDF2-SHA256(password, salt, 600000) and sends only
the key;
-- the server stores verifier = SHA-256(key). Salt = HMAC(PEPPER, username), stored so a PEPPER
change never locks
-- anyone out; unknown usernames get the same HMAC, so the salt endpoint does not reveal which
usernames exist.
CREATE TABLE password_login (
  account_id  INTEGER PRIMARY KEY REFERENCES account(account_id) ON DELETE CASCADE,
  username    TEXT NOT NULL UNIQUE CHECK (length(username) BETWEEN 3 AND 24 AND username NOT GLOB
'*[^a-z0-9_.]*'),
  salt        TEXT NOT NULL CHECK (length(salt) = 32 AND salt NOT GLOB '*[^0-9a-f]*'),
  verifier    TEXT NOT NULL CHECK (length(verifier) = 64 AND verifier NOT GLOB '*[^0-9a-f]*'),
  kdf         TEXT NOT NULL CHECK (kdf IN ('PBKDF2-SHA256-600000')),
  updated_at  TEXT NOT NULL
) STRICT;

-- Sessions: the cookie holds a random token; only its SHA-256 is stored, so a leaked table cannot
be replayed.
CREATE TABLE session (
  token_hash  TEXT PRIMARY KEY CHECK (length(token_hash) = 64 AND token_hash NOT GLOB '*[^0-9a-
f]*'),
  account_id  INTEGER NOT NULL REFERENCES account(account_id) ON DELETE CASCADE,
  created_at  TEXT NOT NULL,
  expires_at  TEXT NOT NULL,
  CHECK (expires_at > created_at)
) STRICT, WITHOUT ROWID;
CREATE INDEX idx_session_account ON session(account_id);

-- Recovery codes replace "forgot password" emails (the Free plan cannot send email). 80 random
bits, so a plain
-- SHA-256 is enough; one active code per account, replaced on every use.
CREATE TABLE recovery_code (
  account_id  INTEGER PRIMARY KEY REFERENCES account(account_id) ON DELETE CASCADE,
```

```
  code_hash   TEXT NOT NULL CHECK (length(code_hash) = 64 AND code_hash NOT GLOB '*[^0-9a-f]*'),
  created_at  TEXT NOT NULL
) STRICT;

-- The student's results: the same facts the phone keeps (FR-101..105), now kept for them across
devices.
CREATE TABLE student_profile (
  account_id     INTEGER PRIMARY KEY REFERENCES account(account_id) ON DELETE CASCADE,
  stream_code    TEXT    NOT NULL REFERENCES stream(stream_code),
  district_code  TEXT    NOT NULL REFERENCES district(district_code),
  z_e4           INTEGER NOT NULL CHECK (z_e4 BETWEEN -40000 AND 40000),
  updated_at     TEXT    NOT NULL
) STRICT;
CREATE INDEX idx_student_profile_stream ON student_profile(stream_code);
CREATE INDEX idx_student_profile_district ON student_profile(district_code);

-- Exactly three subjects: position 1..3 caps it at three in the database; the API refuses fewer.
CREATE TABLE student_subject (
  account_id    INTEGER NOT NULL REFERENCES student_profile(account_id) ON DELETE CASCADE,
  position      INTEGER NOT NULL CHECK (position BETWEEN 1 AND 3),
  subject_code  TEXT    NOT NULL REFERENCES subject(subject_code),
  grade         TEXT    NOT NULL CHECK (grade IN ('A','B','C','S')),
  PRIMARY KEY (account_id, position),
  UNIQUE (account_id, subject_code)
) STRICT, WITHOUT ROWID;
CREATE INDEX idx_student_subject_subject ON student_subject(subject_code);

CREATE TABLE student_achievement (
  achievement_id  INTEGER PRIMARY KEY,
  account_id      INTEGER NOT NULL REFERENCES account(account_id) ON DELETE CASCADE,
  position        INTEGER NOT NULL CHECK (position BETWEEN 1 AND 30),
  activity        TEXT    NOT NULL CHECK (length(trim(activity)) BETWEEN 1 AND 80),
  kind            TEXT    NOT NULL CHECK (kind IN ('SPORT','COMPETITION','CLUB','ARTS','OTHER')),
  level           TEXT    NOT NULL CHECK (level IN
('SCHOOL','ZONAL','DISTRICT','PROVINCIAL','NATIONAL','INTERNATIONAL')),
  place           TEXT    NOT NULL CHECK (place IN ('FIRST','SECOND','THIRD','TAKING_PART')),
  year            INTEGER NOT NULL CHECK (year BETWEEN 2000 AND 2100),
  UNIQUE (account_id, position)
) STRICT;

-- Mirrors shared/account.ts INTERESTS (pinned by worker/account.test.ts).
CREATE TABLE student_interest (
  account_id  INTEGER NOT NULL REFERENCES account(account_id) ON DELETE CASCADE,
  interest    TEXT    NOT NULL CHECK (interest IN ('Building apps','Maths','Science and
labs','Working with people',
                'Business and money','Teaching','Health and caring','Art and
design','Languages','Law and society',
                'Nature and farming','Making and engineering','Sport')),
  PRIMARY KEY (account_id, interest)
) STRICT, WITHOUT ROWID;

-- Only student accounts carry student data (enforced here as well as in the API).
CREATE TRIGGER trg_student_profile_role BEFORE INSERT ON student_profile
WHEN (SELECT role FROM account WHERE account_id = NEW.account_id) IS NOT 'STUDENT'
BEGIN SELECT RAISE(ABORT, 'student_profile requires a STUDENT account'); END;
CREATE TRIGGER trg_student_achievement_role BEFORE INSERT ON student_achievement
WHEN (SELECT role FROM account WHERE account_id = NEW.account_id) IS NOT 'STUDENT'
BEGIN SELECT RAISE(ABORT, 'student_achievement requires a STUDENT account'); END;
CREATE TRIGGER trg_student_interest_role BEFORE INSERT ON student_interest
WHEN (SELECT role FROM account WHERE account_id = NEW.account_id) IS NOT 'STUDENT'
BEGIN SELECT RAISE(ABORT, 'student_interest requires a STUDENT account'); END;
```

Appendix D: Contact details (migration 0007)

Reproduced verbatim from `migrations/0007_student_contact.sql`.

```
-- ZedPath D1 migration 0007: contact details, compulsory at sign-up (CR-003, product owner's
decision 2026-10-11:
-- "account creation should be compulsory at start, with name, email, mobile number" and the
birthday moved to it).
-- Design: ZP-DOC-05 Section "Contact details (CR-003)". This REPLACES the CR-001 promise "a
username only": the app,
-- README and business case now say exactly what is collected and why.
-- One row per account, in its own table: personal data kept apart from the identity table (shared
by every role) and
-- from the student's results, so it can be audited, exported or deleted on its own. Deleted with
the account.
-- Email and mobile are NOT unique: siblings may share a parent's number, and a uniqueness error
would reveal that a
-- number or address is registered. They are NOT verified yet (no sending domain, no SMS service):
the *_verified_at
-- columns stay NULL until verification exists, and nothing may rely on them before then.
CREATE TABLE student_contact (
  account_id         INTEGER PRIMARY KEY REFERENCES account(account_id) ON DELETE CASCADE,
  full_name          TEXT NOT NULL CHECK (length(trim(full_name)) BETWEEN 2 AND 80),
  email              TEXT NOT NULL CHECK (length(email) <= 254 AND email = lower(email) AND email
GLOB '?*@?*.?*' AND email NOT GLOB '* *'),
  mobile             TEXT NOT NULL CHECK (mobile GLOB '+947[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-
9]'),   -- Sri Lankan mobile, E.164
  birth_date         TEXT NOT NULL CHECK (birth_date GLOB '[12][0-9][0-9][0-9]-[01][0-9]-[0-3][0-9]'
AND date(birth_date) = birth_date),
  consent_version    TEXT NOT NULL CHECK (consent_version IN ('2026-10-11')),    -- the privacy
wording agreed to
  consent_at         TEXT NOT NULL,
  email_verified_at  TEXT,
  mobile_verified_at TEXT,
  updated_at         TEXT NOT NULL
) STRICT;
```

Appendix E: Answer feedback (migration 0008)

Reproduced verbatim from `migrations/0008_answer_feedback.sql`.

```
-- ZedPath D1 migration 0008: feedback on Ask ZedPath answers (CR-004, product owner's GO 2026-10-
11).
-- Not mistake_report (0001): that table reports a stored, cited FACT (fact_id is required); an AI
answer is not one.
-- Privacy: sent only when the student taps "Helpful" or "Report a wrong answer", after the screen
says the question
-- and answer go to the ZedPath team. No account, device or IP is stored with it.
CREATE TABLE answer_feedback (
  feedback_id  INTEGER PRIMARY KEY,
  rating       TEXT NOT NULL CHECK (rating IN ('HELPFUL','WRONG')),
  question     TEXT NOT NULL CHECK (length(question) BETWEEN 2 AND 500),
  answer       TEXT NOT NULL CHECK (length(answer) BETWEEN 1 AND 4000),
  pages        TEXT NOT NULL CHECK (json_valid(pages)),                -- the handbook pages the
answer cited, e.g. [87,151]
  note         TEXT CHECK (note IS NULL OR length(note) <= 500),
  lang         TEXT NOT NULL CHECK (lang IN ('en','si','ta')),
  created_at   TEXT NOT NULL,
  state        TEXT NOT NULL DEFAULT 'OPEN' CHECK (state IN ('OPEN','CHECKED'))
) STRICT;
CREATE INDEX idx_answer_feedback_open ON answer_feedback(created_at) WHERE state = 'OPEN';
```

Appendix F: Journey, avatars and group rules on the account (migration 0010)

Reproduced verbatim from `migrations/0010_journey_avatars_rules.sql`, generated by `tools/seed/cr006-sql.ts` (the 255 offering rows are the official Uni-Code list).

```
-- 0010: CR-006 (Praveen 2026-10-11): group rules accepted once per account, profile avatars, and
the Journey saved on
-- the account (followed courses and notices, ticked steps). Designed in ZP-DOC-05, CR-006.
Generated by
-- tools/seed/cr006-sql.ts; do not edit by hand.

-- 1. The 255 offerings of 2025/26 (handbook pp.142-148). 0001 defined OFFERING but left it empty;
filling it lets a
--    followed course reference a real offering instead of a free-text code.
INSERT INTO offering (course_code, institution_letter) VALUES
  ('001', 'A'),
  ('001', 'B'),
  ('001', 'C'),
  ('001', 'D'),
  ('001', 'E'),
  ('001', 'F'),
  ('001', 'G'),
  ('001', 'H'),
  ('001', 'K'),
  ('001', 'L'),
  ('001', 'M'),
  ('001', 'U'),
  ('002', 'B'),
  ('002', 'C'),
  ('003', 'B'),
  ('004', 'E'),
  ('004', 'H'),
  ('004', 'K'),
  ('004', 'L'),
  ('004', 'M'),
  ('005', 'M'),
  ('006', 'A'),
  ('006', 'B'),
  ('006', 'C'),
  ('006', 'D'),
  ('006', 'E'),
  ('006', 'F'),
  ('006', 'H'),
  ('006', 'J'),
  ('007', 'K'),
  ('007', 'L'),
  ('007', 'R'),
  ('008', 'B'),
  ('008', 'C'),
  ('008', 'E'),
  ('008', 'F'),
  ('008', 'G'),
  ('008', 'J'),
  ('009', 'G'),
  ('010', 'G'),
  ('011', 'G'),
  ('012', 'C'),
  ('012', 'D'),
  ('012', 'E'),
  ('012', 'F'),
  ('012', 'T'),
  ('012', 'W'),
  ('013', 'A'),
  ('013', 'B'),
  ('013', 'C'),
  ('013', 'D'),
```

```
  ('013', 'E'),
  ('013', 'F'),
  ('013', 'H'),
  ('013', 'J'),
  ('014', 'L'),
  ('015', 'K'),
  ('015', 'L'),
  ('015', 'M'),
  ('015', 'R'),
  ('015', 'W'),
  ('016', 'A'),
  ('016', 'B'),
  ('016', 'C'),
  ('016', 'D'),
  ('016', 'E'),
  ('016', 'F'),
  ('016', 'H'),
  ('016', 'J'),
  ('016', 'K'),
  ('016', 'L'),
  ('016', 'M'),
  ('017', 'C'),
  ('018', 'C'),
  ('018', 'D'),
  ('018', 'E'),
  ('018', 'H'),
  ('018', 'J'),
  ('019', 'A'),
  ('019', 'B'),
  ('019', 'C'),
  ('019', 'D'),
  ('019', 'E'),
  ('019', 'F'),
  ('019', 'H'),
  ('019', 'J'),
  ('019', 'K'),
  ('020', 'S'),
  ('021', 'L'),
  ('022', 'R'),
  ('022', 'W'),
  ('023', 'G'),
  ('024', 'G'),
  ('025', 'A'),
  ('025', 'B'),
  ('025', 'E'),
  ('026', 'G'),
  ('027', 'D'),
  ('028', 'C'),
  ('029', 'W'),
  ('030', 'G'),
  ('031', 'D'),
  ('032', 'A'),
  ('032', 'P'),
  ('033', 'A'),
  ('034', 'G'),
  ('035', 'B'),
  ('035', 'C'),
  ('035', 'L'),
  ('036', 'E'),
  ('036', 'W'),
  ('037', 'A'),
  ('037', 'B'),
  ('037', 'C'),
  ('037', 'E'),
  ('037', 'F'),
  ('037', 'H'),
  ('038', 'K'),
  ('038', 'R'),
  ('039', 'B'),
  ('041', 'S'),
  ('050', 'K'),
  ('051', 'B'),
```

```
  ('051', 'C'),
  ('051', 'E'),
  ('051', 'F'),
  ('052', 'B'),
  ('052', 'C'),
  ('052', 'E'),
  ('052', 'F'),
  ('053', 'B'),
  ('054', 'A'),
  ('054', 'B'),
  ('055', 'D'),
  ('056', 'G'),
  ('057', 'G'),
  ('058', 'A'),
  ('059', 'A'),
  ('060', 'B'),
  ('062', 'F'),
  ('063', 'J'),
  ('064', 'U'),
  ('065', 'U'),
  ('066', 'U'),
  ('067', 'U'),
  ('068', 'C'),
  ('068', 'E'),
  ('068', 'Y'),
  ('068', 'Z'),
  ('069', 'E'),
  ('069', 'Y'),
  ('069', 'Z'),
  ('070', 'E'),
  ('071', 'Y'),
  ('071', 'Z'),
  ('072', 'Y'),
  ('073', 'U'),
  ('075', 'U'),
  ('076', 'U'),
  ('077', 'C'),
  ('079', 'J'),
  ('081', 'E'),
  ('081', 'L'),
  ('082', 'C'),
  ('082', 'L'),
  ('083', 'D'),
  ('084', 'J'),
  ('085', 'Z'),
  ('086', 'B'),
  ('087', 'M'),
  ('088', 'U'),
  ('090', 'U'),
  ('091', 'G'),
  ('092', 'K'),
  ('092', 'L'),
  ('093', 'F'),
  ('094', 'F'),
  ('095', 'F'),
  ('096', 'C'),
  ('096', 'L'),
  ('096', 'T'),
  ('097', 'G'),
  ('098', 'D'),
  ('098', 'E'),
  ('098', 'H'),
  ('098', 'L'),
  ('099', 'C'),
  ('099', 'D'),
  ('099', 'L'),
  ('100', 'D'),
  ('101', 'R'),
  ('102', 'A'),
  ('102', 'C'),
  ('102', 'D'),
  ('102', 'E'),
```

```
  ('102', 'F'),
  ('102', 'K'),
  ('102', 'L'),
  ('102', 'M'),
  ('102', 'U'),
  ('103', 'A'),
  ('103', 'C'),
  ('103', 'D'),
  ('103', 'E'),
  ('103', 'F'),
  ('103', 'H'),
  ('103', 'J'),
  ('103', 'K'),
  ('103', 'L'),
  ('103', 'M'),
  ('103', 'U'),
  ('104', 'A'),
  ('104', 'C'),
  ('104', 'D'),
  ('104', 'F'),
  ('104', 'H'),
  ('104', 'J'),
  ('104', 'K'),
  ('104', 'R'),
  ('104', 'U'),
  ('105', 'C'),
  ('105', 'D'),
  ('105', 'L'),
  ('106', 'F'),
  ('107', 'L'),
  ('108', 'C'),
  ('108', 'D'),
  ('109', 'G'),
  ('110', 'D'),
  ('111', 'B'),
  ('112', 'B'),
  ('112', 'C'),
  ('113', 'F'),
  ('114', 'U'),
  ('115', 'D'),
  ('116', 'C'),
  ('117', 'G'),
  ('118', 'D'),
  ('119', 'D'),
  ('127', 'D'),
  ('128', 'C'),
  ('129', 'C'),
  ('130', 'C'),
  ('131', 'C'),
  ('132', 'U'),
  ('133', 'R'),
  ('134', 'C'),
  ('135', 'U'),
  ('136', 'L'),
  ('137', 'A'),
  ('138', 'A'),
  ('139', 'C'),
  ('141', 'D'),
  ('142', 'F'),
  ('143', 'B');

-- 2. Group rules: accepted once per account, not once per group. A fact about the account and a
rules version, so it
--    is its own relation (one row per account per version accepted: a new version asks again and
the history stays).
CREATE TABLE group_rules_acceptance (
  account_id     INTEGER NOT NULL REFERENCES account(account_id) ON DELETE CASCADE,
  rules_version  TEXT    NOT NULL CHECK (rules_version IN ('2026-10-11')),
  accepted_at    TEXT    NOT NULL,
  PRIMARY KEY (account_id, rules_version)
) STRICT, WITHOUT ROWID;
```

```
-- Carry over what members already accepted under 0009, at the time they first joined.
group_member.rules_version is no
-- longer read or written (expand, then contract): it stays until this release is stable, so a
rollback to the previous
-- version still works, and a later migration drops it.
INSERT OR IGNORE INTO group_rules_acceptance (account_id, rules_version, accepted_at)
  SELECT account_id, rules_version, min(joined_at) FROM group_member WHERE rules_version IS NOT NULL
GROUP BY account_id, rules_version;

-- 3. Avatar: one preset picture per account (no photo upload). NULL = initials, the behaviour
before CR-006.
ALTER TABLE account ADD COLUMN avatar TEXT CHECK (avatar IS NULL OR avatar IN
('a01','a02','a03','a04','a05','a06','a07','a08','a09','a10','a11','a12'));

-- 4. Journey on the account (it lived only on the phone before CR-006).
-- 4a. Courses the student follows. Composite foreign key to OFFERING.
CREATE TABLE journey_course (
  account_id          INTEGER NOT NULL REFERENCES account(account_id) ON DELETE CASCADE,
  course_code         TEXT    NOT NULL,
  institution_letter  TEXT    NOT NULL,
  followed_at         TEXT    NOT NULL,
  PRIMARY KEY (account_id, course_code, institution_letter),
  FOREIGN KEY (course_code, institution_letter) REFERENCES offering(course_code, institution_letter)
) STRICT, WITHOUT ROWID;
-- 4b. Notices and other paths the student follows (Gazette exams, intakes). The route catalogue is
curated read-model
--     data compiled into the Worker (ZP-DOC-05, read-model decision), so the id is checked by shape
here and against
--     the catalogue by the Worker. applied_at = the student ticked "I applied"; the Worker allows
it only once the
--     notice is open.
CREATE TABLE journey_notice (
  account_id   INTEGER NOT NULL REFERENCES account(account_id) ON DELETE CASCADE,
  route_id     TEXT    NOT NULL CHECK (length(route_id) = 4 AND route_id GLOB 'R[0-9][0-9][0-9]'),
  followed_at  TEXT    NOT NULL,
  applied_at   TEXT,
  CHECK (applied_at IS NULL OR applied_at >= followed_at),
  PRIMARY KEY (account_id, route_id)
) STRICT, WITHOUT ROWID;
-- 4c. Journey steps the student ticked. The step list is fixed (shared/journey.ts); a step can be
ticked only once its
--     window is open, which the Worker checks against the dated read model before writing.
CREATE TABLE journey_step (
  account_id  INTEGER NOT NULL REFERENCES account(account_id) ON DELETE CASCADE,
  step_id     TEXT    NOT NULL CHECK (step_id IN
('RESULTS','LIST','COMPARE','UGC_APPLY','UGC_ORDER','APTITUDE','UGC_SELECTION','UGC_REGISTER')),
  done_at     TEXT    NOT NULL,
  PRIMARY KEY (account_id, step_id)
) STRICT, WITHOUT ROWID;
```

Appendix G: Rooms an account opened (migration 0012)

Reproduced verbatim from `migrations/0012_group_room_seen.sql`.

```
-- 0012: security audit run-1, lead 3 (Praveen 2026-10-15: "do all hardening"). Deleting an account
must remove its
-- messages from every group room it used, including rooms it later left. Messages live in each
room's Durable Object,
-- outside D1's cascade, so the Worker needs to know which rooms to purge. Designed in ZP-DOC-05
(lead 3).
--
-- One row per account per room it opened (written when the live chat connects). It is deleted with
the account or the
-- group (CASCADE); the Worker reads it, with current memberships, BEFORE deleting the account.
CREATE TABLE group_room_seen (
  account_id  INTEGER NOT NULL REFERENCES account(account_id) ON DELETE CASCADE,
  group_id    INTEGER NOT NULL REFERENCES course_group(group_id) ON DELETE CASCADE,
  first_at    TEXT    NOT NULL,
  PRIMARY KEY (account_id, group_id)
) STRICT, WITHOUT ROWID;
-- Rooms a member is in today were opened by them or not; seed with current memberships so nothing
is missed.
INSERT OR IGNORE INTO group_room_seen (account_id, group_id, first_at) SELECT account_id, group_id,
joined_at FROM group_member;
```

Appendix H: Demo accounts (migration 0011)

Reproduced verbatim from `migrations/0011_demo_accounts.sql`.

```
-- 0011: CR-007 (Praveen 2026-10-15): "Try the demo" on the Welcome screen. Each tap makes a private throwaway account
-- with the sample student, deleted 24 hours later. Designed in ZP-DOC-05, CR-007.
--
-- A demo account is an ordinary ACCOUNT with an expiry: no new table, so every child row (profile, list, journey,
-- sessions, memberships) already cascades when it is deleted. NULL = a real account, the behaviour before CR-007.
ALTER TABLE account ADD COLUMN demo_expires_at TEXT CHECK (demo_expires_at IS NULL OR demo_expires_at > created_at);

-- Cleanup finds expired demo accounts without scanning real ones (partial index: real accounts are not in it).
CREATE INDEX idx_account_demo_expires ON account(demo_expires_at) WHERE demo_expires_at IS NOT NULL;
```

Appendix I: Group rules (migration 0009)

Reproduced verbatim from `migrations/0009_group_rules.sql`.

```
-- 0009: group rules (CR-005, video 2 2026-10-11: "Groups should have rules too"). Designed in ZP-
DOC-05, CR-005.
-- A member accepts the group rules before taking part; the accepted version is recorded on the
membership, so a new
-- version of the rules can be asked for again. NULL = joined before rules existed (or not yet
accepted): the Worker
-- then keeps the live room closed until the member accepts (enforced in worker/groups.ts, not only
on the phone).
-- Reuses group_member (one row per member per group) instead of a new table: acceptance is a fact
about that row.
ALTER TABLE group_member ADD COLUMN rules_version TEXT CHECK (rules_version IS NULL OR rules_version
IN ('2026-10-11'));
```

Appendix J: Groups (migration 0006)

Reproduced verbatim from `migrations/0006_groups.sql`. Migration 0005 inserts the courses and institutions and is generated by `tools/seed/courses-sql.ts`.

```
-- ZedPath D1 migration 0006: course groups, verified-senior invites, reports and blocks (CR-002,
Praveen's GO
-- 2026-10-11). Design: ZP-DOC-05 Section "Groups". Messages are NOT stored here: each group's
messages live in that
-- group's own Durable Object (GroupRoom, SQLite storage), so a busy chat never touches this
database.
-- Privacy (NFR-030): still a username only. Deleting an account deletes its memberships, reports,
blocks and any
-- redeemed invite (ON DELETE CASCADE); the Worker also tells each of its groups to delete that
account's messages.

-- A verified senior is a SENIOR account (the role and account.verified_for_letter already exist,
migration 0003).
-- Praveen checks a senior himself and hands over a one-time code; only the code's SHA-256 is
stored. Redeeming it at
-- sign-up creates the SENIOR account and marks the invite in one atomic batch.
CREATE TABLE senior_invite (
  invite_id           INTEGER PRIMARY KEY,
  code_hash           TEXT    NOT NULL UNIQUE CHECK (length(code_hash) = 64 AND code_hash NOT GLOB
'*[^0-9a-f]*'),
  course_code         TEXT    NOT NULL REFERENCES course(course_code),
  institution_letter  TEXT    NOT NULL REFERENCES institution(letter),
  study_year          INTEGER NOT NULL CHECK (study_year BETWEEN 1 AND 6),
  created_at          TEXT    NOT NULL,
  expires_at          TEXT    NOT NULL,
  redeemed_by         INTEGER UNIQUE REFERENCES account(account_id) ON DELETE CASCADE,   -- one
invite per account
  redeemed_at         TEXT,
  CHECK (expires_at > created_at),
  CHECK ((redeemed_by IS NULL) = (redeemed_at IS NULL))
) STRICT;

-- An invite can only be redeemed by a SENIOR account verified for the invite's institution, and
never twice.
CREATE TRIGGER trg_senior_invite_redeem BEFORE UPDATE OF redeemed_by ON senior_invite
WHEN NEW.redeemed_by IS NOT NULL AND (
  OLD.redeemed_by IS NOT NULL
  OR (SELECT role FROM account WHERE account_id = NEW.redeemed_by) IS NOT 'SENIOR'
  OR (SELECT verified_for_letter FROM account WHERE account_id = NEW.redeemed_by) IS NOT
NEW.institution_letter)
BEGIN SELECT RAISE(ABORT, 'senior_invite: invalid redemption'); END;

-- A group is either one target course (all universities) or one A/L batch (stream + district).
Created on first join.
CREATE TABLE course_group (
  group_id       INTEGER PRIMARY KEY,
  kind           TEXT NOT NULL CHECK (kind IN ('COURSE','BATCH')),
  course_code    TEXT REFERENCES course(course_code),
  stream_code    TEXT REFERENCES stream(stream_code),
  district_code  TEXT REFERENCES district(district_code),
  created_at     TEXT NOT NULL,
  CHECK ((kind = 'COURSE') = (course_code IS NOT NULL)),
  CHECK ((kind = 'BATCH') = (stream_code IS NOT NULL AND district_code IS NOT NULL)),
  CHECK (kind = 'BATCH' OR (stream_code IS NULL AND district_code IS NULL))
) STRICT;
-- Partial unique indexes, not UNIQUE(kind, course, stream, district): SQLite treats NULLs as
distinct in UNIQUE.
CREATE UNIQUE INDEX ux_course_group_course ON course_group(course_code) WHERE kind = 'COURSE';
CREATE UNIQUE INDEX ux_course_group_batch  ON course_group(stream_code, district_code) WHERE kind =
'BATCH';

CREATE TABLE group_member (
  group_id    INTEGER NOT NULL REFERENCES course_group(group_id) ON DELETE CASCADE,
```

```
  account_id  INTEGER NOT NULL REFERENCES account(account_id) ON DELETE CASCADE,
  joined_at   TEXT    NOT NULL,
  PRIMARY KEY (group_id, account_id)
) STRICT, WITHOUT ROWID;
CREATE INDEX idx_group_member_account ON group_member(account_id);

-- One report per member per message. message_id is the GroupRoom's id for the message (128 random
bits, hex).
CREATE TABLE message_report (
  group_id     INTEGER NOT NULL REFERENCES course_group(group_id) ON DELETE CASCADE,
  message_id   TEXT    NOT NULL CHECK (length(message_id) = 32 AND message_id NOT GLOB '*[^0-9a-
f]*'),
  reporter_id  INTEGER NOT NULL REFERENCES account(account_id) ON DELETE CASCADE,
  reason       TEXT    NOT NULL CHECK (reason IN ('HARMFUL','WRONG_INFO','SPAM','OTHER')),
  created_at   TEXT    NOT NULL,
  PRIMARY KEY (group_id, message_id, reporter_id)
) STRICT, WITHOUT ROWID;
CREATE INDEX idx_message_report_reporter ON message_report(reporter_id);

-- Blocks hide the blocked account's messages from the blocker, in every group.
CREATE TABLE account_block (
  blocker_id  INTEGER NOT NULL REFERENCES account(account_id) ON DELETE CASCADE,
  blocked_id  INTEGER NOT NULL REFERENCES account(account_id) ON DELETE CASCADE,
  created_at  TEXT    NOT NULL,
  PRIMARY KEY (blocker_id, blocked_id),
  CHECK (blocker_id <> blocked_id)
) STRICT, WITHOUT ROWID;
CREATE INDEX idx_account_block_blocked ON account_block(blocked_id);
```

Appendix K: Glossary

Table 15: Terms used in this document

Term	Meaning
BCNF	Boyce-Codd normal form: every determinant is a candidate key
Category (union type)	An EER subclass of the union of several superclasses
D1	Cloudflare's serverless SQL database, built on SQLite
Identifying relationship	The relationship that links a weak entity to the owner that identifies it
MVD	Multi-valued dependency, the basis of fourth normal form (4NF)
Read model	A query-optimised copy of data derived from the system of record
STRICT table	An SQLite table that rejects values of the wrong type
Weak entity	An entity that cannot be identified by its own attributes alone

Appendix L: References

Table 16: Sources referred to in this document

Ref	Source
[1]	R. Elmasri and S. Navathe, *Fundamentals of Database Systems*, 7th edition, Pearson, 2016 (chapters 3, 4, 9, 14, 15)
[2]	Cloudflare, *D1: Define foreign keys* and *SQL statements*, developers.cloudflare.com, retrieved 5 October 2026
[3]	Cloudflare, *D1 limits and pricing*, developers.cloudflare.com, retrieved 5 October 2026
[4]	SQLite, *STRICT Tables* and *Generated Columns*, sqlite.org
[5]	UGC handbook and cut-off table 2025/26 (see `data/ugc/PROVENANCE.md`)
[6]	ZP-DOC-03 Software Requirements Specification